

Persistence (I)

I/O and Files

中国人民大学信息学院

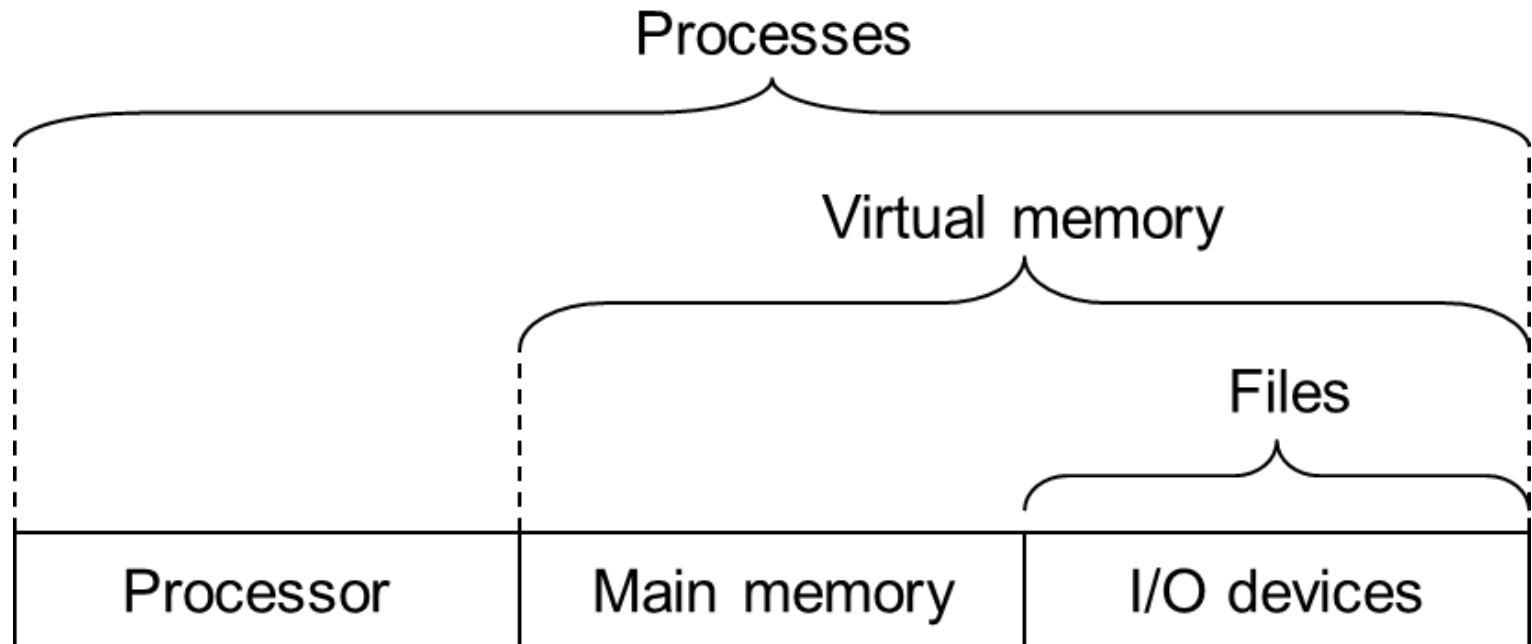
卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

OS: Three Easy Pieces

- 虚拟化
- 持久化
- 并发



Outline

- I/O Devices
- Unix I/O
- Reading File Metadata
- Sharing Files & I/O redirection

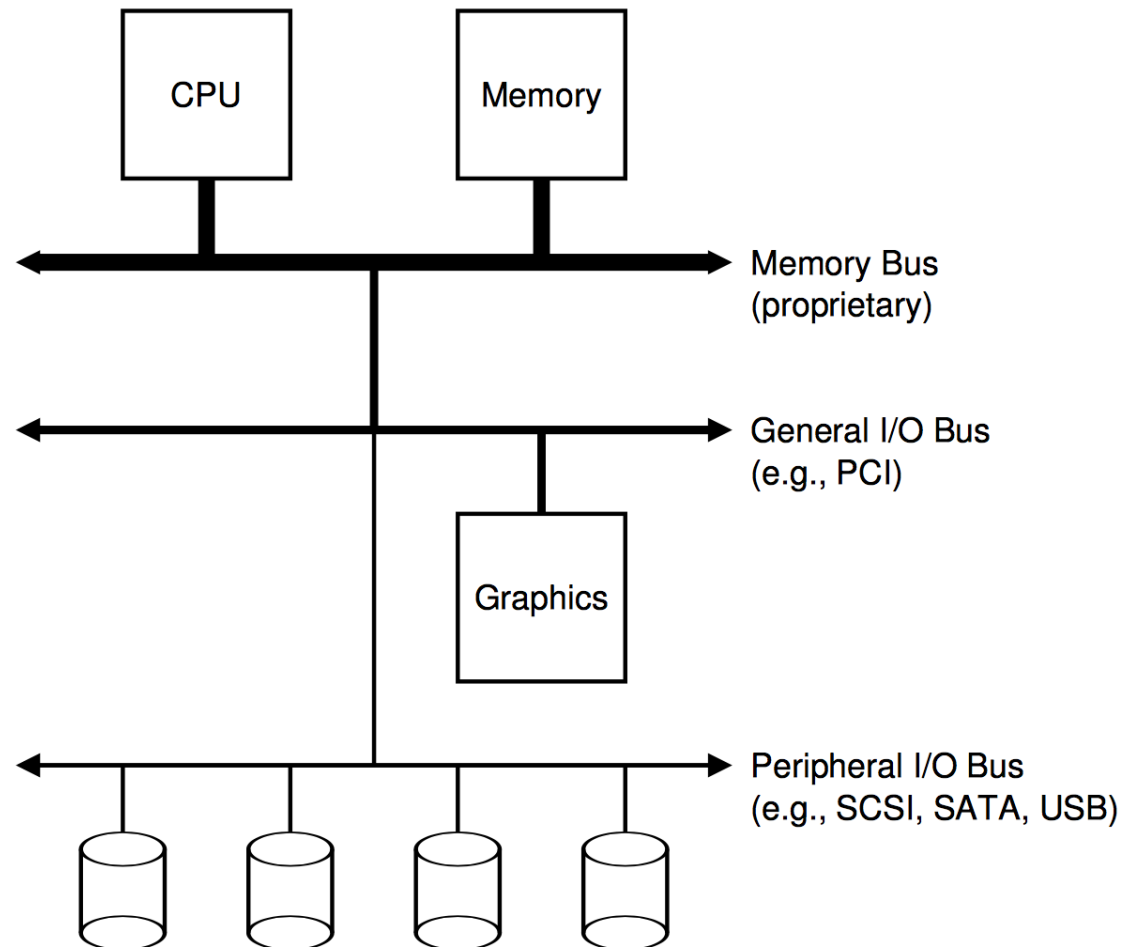
I/O Devices

- I/O: Input/Output
 - 持久化设备
 - 加载程序、数据
 - 保存状态
 - CPU, Cache, RAM都无法保存状态
 - 新的挑战者
 - 非易失内存
 - Non-Volatile Memory
 - Storage Class Memory
 - Persistent Memory

I/O Devices

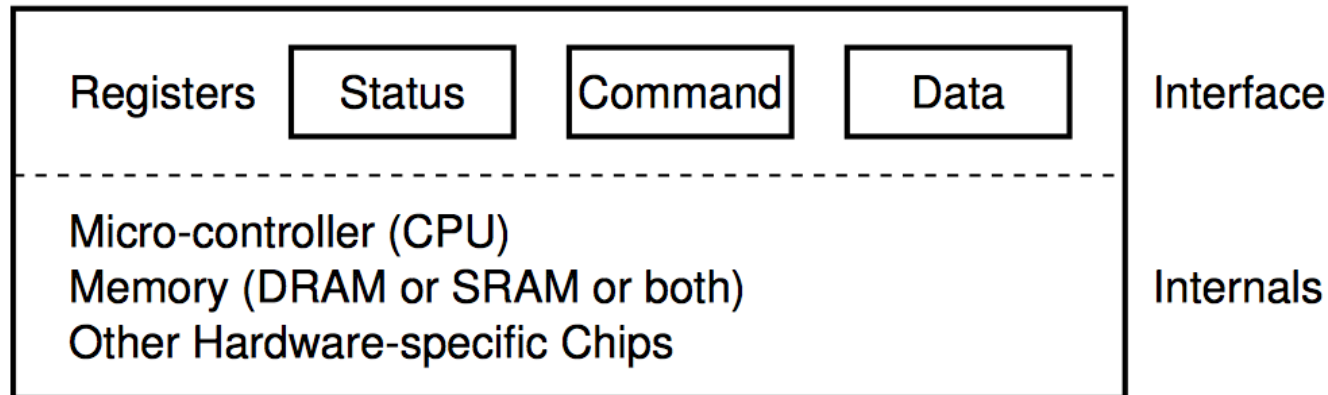
- System Architecture

- 层次结构原因
 - 快速设备近
 - 慢速设备远



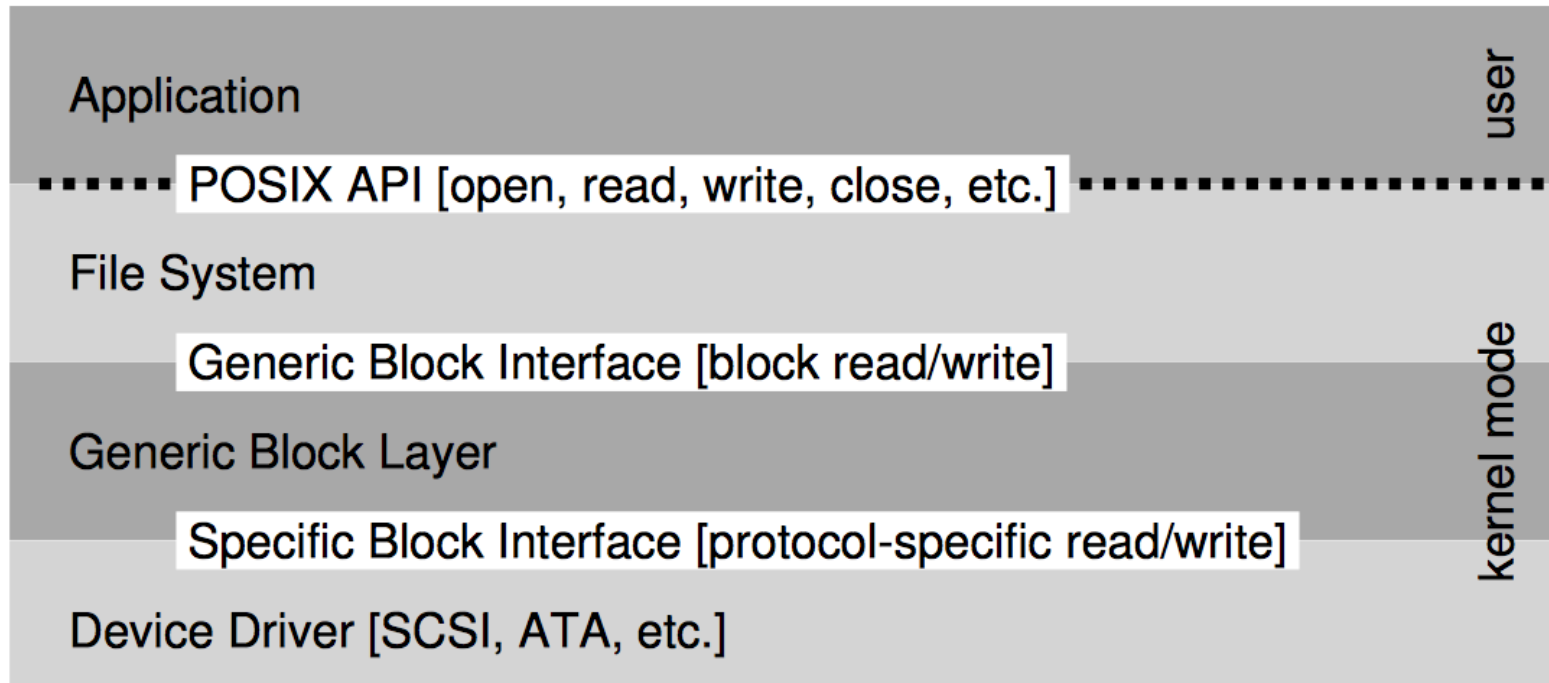
I/O Devices

- I/O设备的通用结构
 - Status
 - Command
 - Data



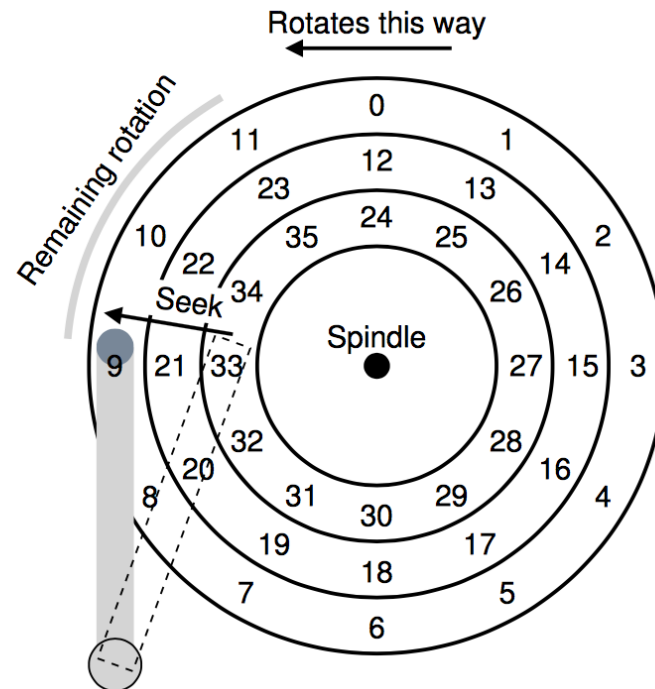
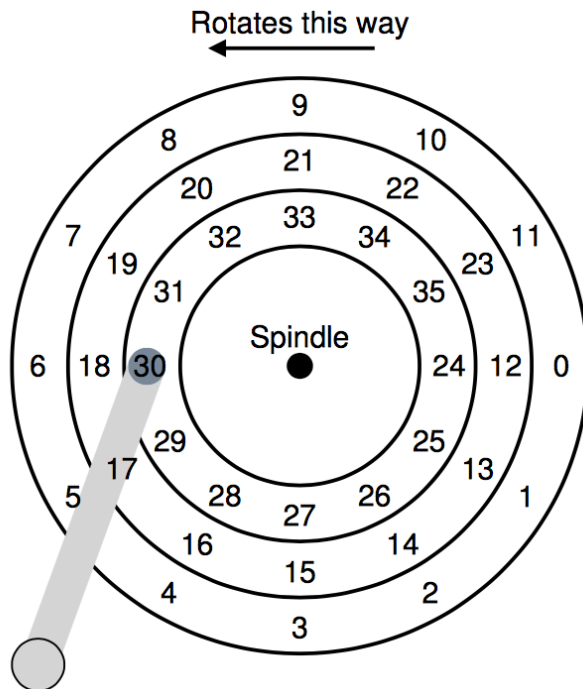
I/O Devices

- File System Stack



I/O Devices

- 磁盘
- 闪存



I/O Devices

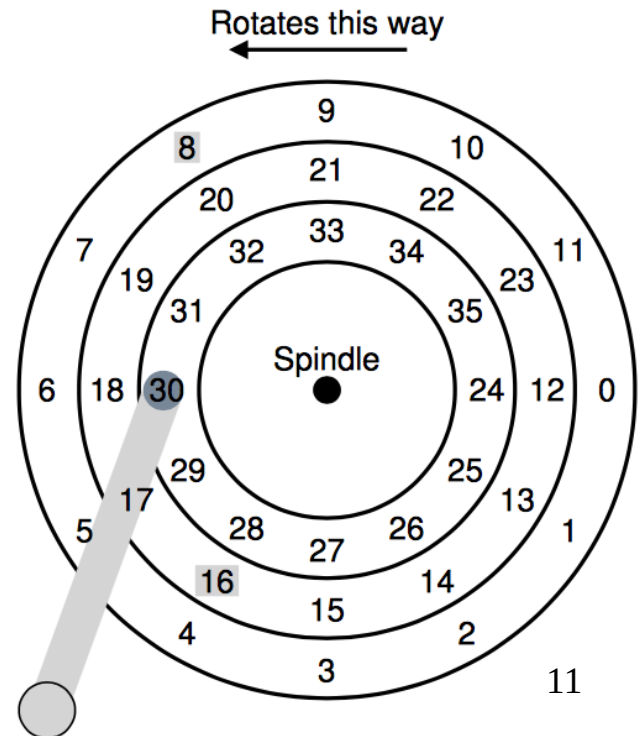
- 磁盘调度
 - : **Shortest Seek Time First**
 - picking requests on the nearest track first
 - 问题
 - 饥饿
 - 陷入局部最优，达不到全局最优

I/O Devices

- **Elevator (a.k.a. SCAN or C-SCAN)**
 - 外->内; 内->外; 外->内...
 - **F-SCAN**
 - Freeze the queue, 一个方向扫完之后才处理新来的请求
 - 避免饥饿
 - 效率问题: 新来的附近请求
 - **C-SCAN (Circular SCAN)**
 - 从外到内移动磁头, 移动到目前已有请求中最内道请求的位置, 再置位到已有请求中最外道的位置
 - 所有磁道更加公平, 否则中间磁道访问的机会更多

I/O Devices

- **SSTF**和**Elevator**算法的通用问题
 - 没有考虑到旋转的影响
- **SPTF: Shortest Positioning Time First**
 - 当前磁头在30
 - 下一步到16, 还是8?
 - It depends
 - 看**Seek time**和**rotation delay**哪个更大, 最好在**Disk controller**里面实现



课堂练习

• 磁盘调度算法

三、某移动臂磁盘的柱面由外向里顺序编号，假定当前磁头停在 100 号柱面且移动臂方向是向里的，现有如下

表 1 所示的请求序列在等待访问磁盘：

表 1 访问磁盘请求序列

请求次序	1	2	3	4	5	6	7	8	9	10
柱面号	190	10	160	80	90	125	30	20	140	25

回答下面的问题：

(F-SCAN)

① 写出分别采用“最短查找时间优先算法”和“电梯调度算法”时，实际处理上述请求的次序。

② 针对本题比较上述两种算法，就移动臂所花的时间（忽略移动臂改向时间）而言，哪种算法更合适？简

要说明之。

练习答案

- 最短查找时间优先
 - 100-90-80-125-140-160-190-30-25-20-10
 - $10+10+45+15+20+30+160+5+5+10 = 310$
- 电梯算法 (F-SCAN)
 - 100-125-140-160-190-90-80-30-25-20-10
 - $25+15+20+30+100+10+50+5+5+10 = 270$

I/O Devices

- RAID

- 1988 年美国加州大学伯克利分校的 D. A. Patterson 教授等首次在论文 “A Case of Redundant Array of Inexpensive Disks” 中提出了 RAID 概念，即廉价冗余磁盘阵列（Redundant Array of Inexpensive Disks）
- 后来 RAID 咨询委员会决定用“独立”替代“廉价”，于时 RAID 变成了独立磁盘冗余阵列（Redundant Array of Independent Disks）

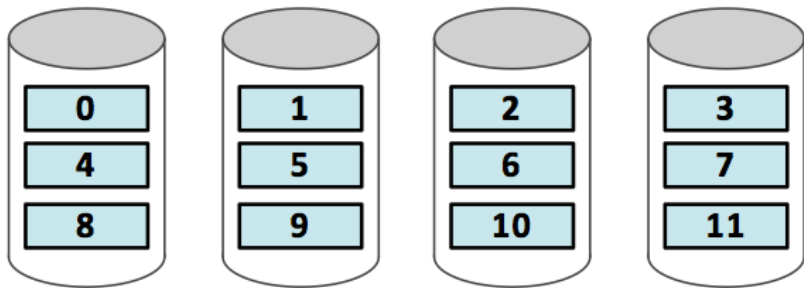


图灵奖获得者
David Patterson

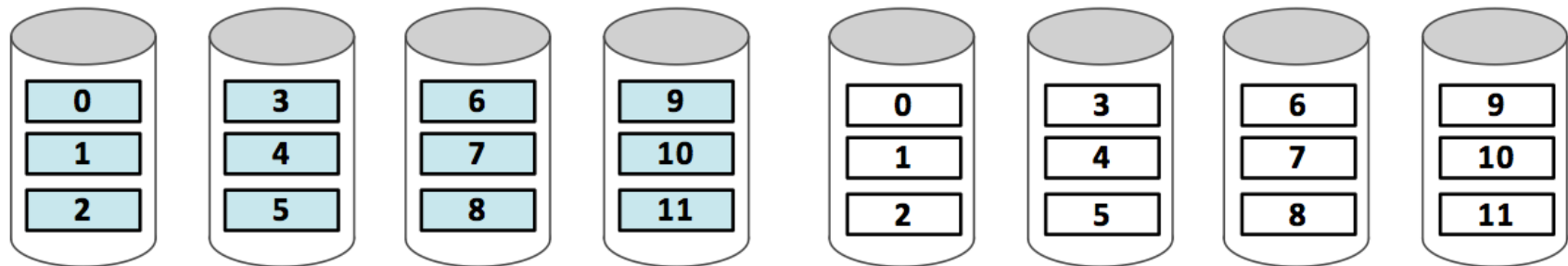
I/O Devices

- 无冗余 (**RAID 0**)
 - 条带化 (**striping**)
- 镜像 (**RAID 1**)
- 块交叉奇偶校验 (**RAID 5**)
- **RAID 10/RAID 01**
- **JBOD**

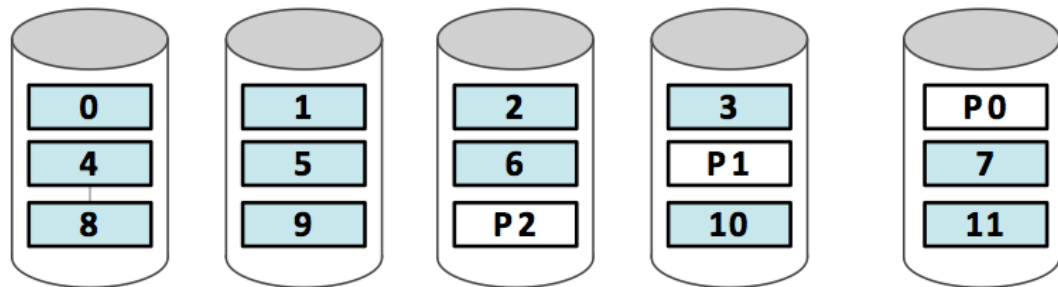
RAID 0



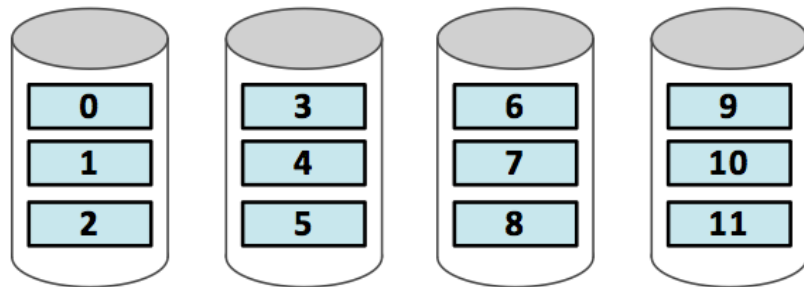
RAID 1



RAID 5



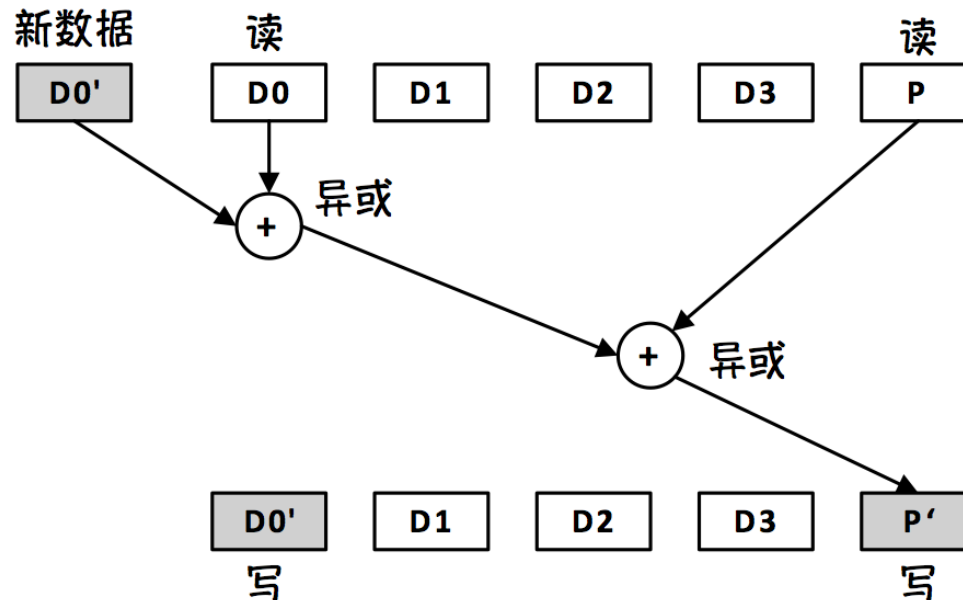
JBOD



I/O Devices

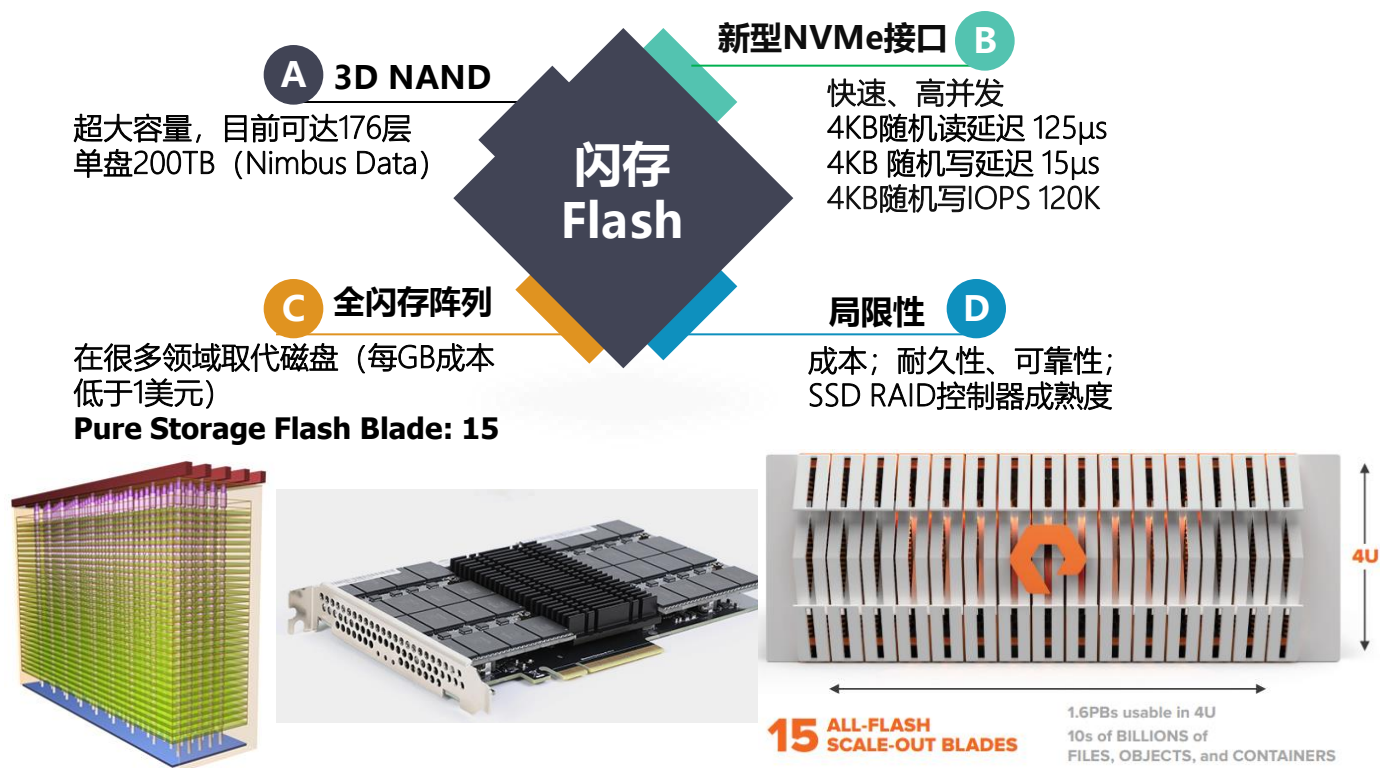
- RAID5

- 四份原始数据分别是0、1、1、0，采用偶校验，校验位P是0 ($=0 \oplus 1 \oplus 1 \oplus 0$)
- 当D0从0修改为1后，新校验位 $P'=D0' \oplus D1 \oplus D2 \oplus D3$



I/O Devices

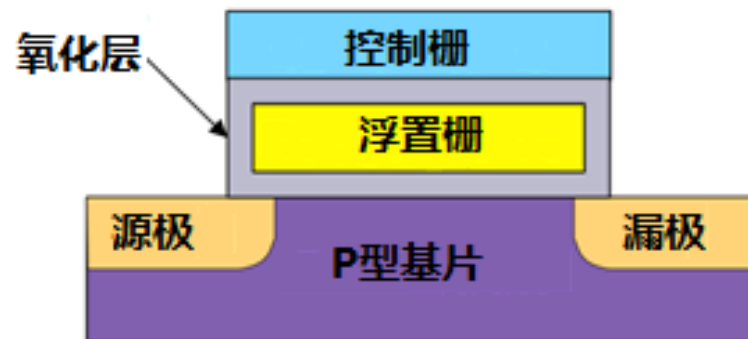
- Flash-based SSD



I/O Devices

- Flash-based SSD

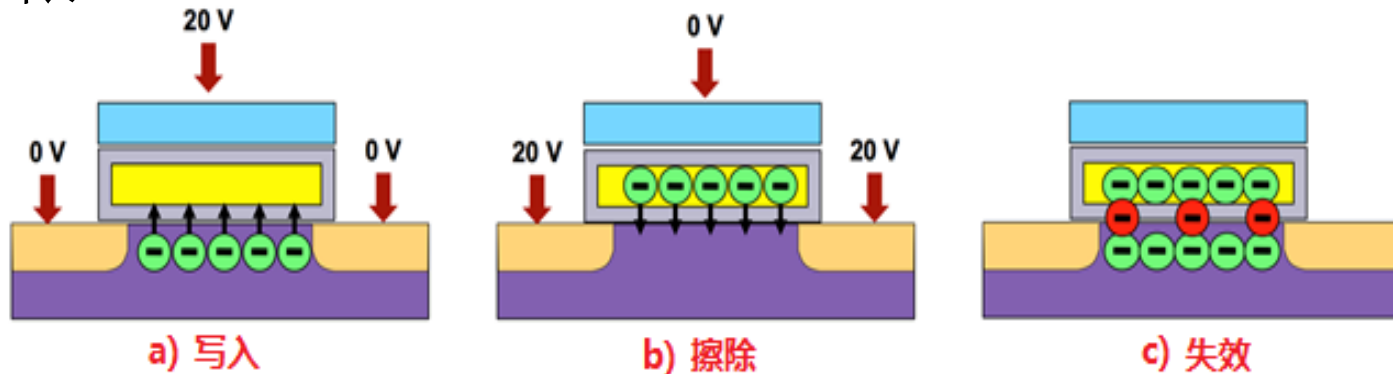
- Flash: 与E2PROM类似，利用捕获电子存储信息
- 闪存的基本存储单元由P型基片、源极、漏极、控制栅和浮置栅等构成



I/O Devices

- 闪存操作接口

- 读、写、擦除
- 耐久性问题：由于氧化层很薄，允许擦除的次数存在上限。



		iiii	Initial: pages in block are invalid (i)
Erase()	→	EEEE	State of pages in block set to erased (E)
Program(0)	→	VEEE	Program page 0; state set to valid (V)
Program(0)	→	error	Cannot re-program page after programming
Program(1)	→	VVEE	Program page 1
Erase()	→	EEEE	Contents erased; all pages programmable

I/O Devices

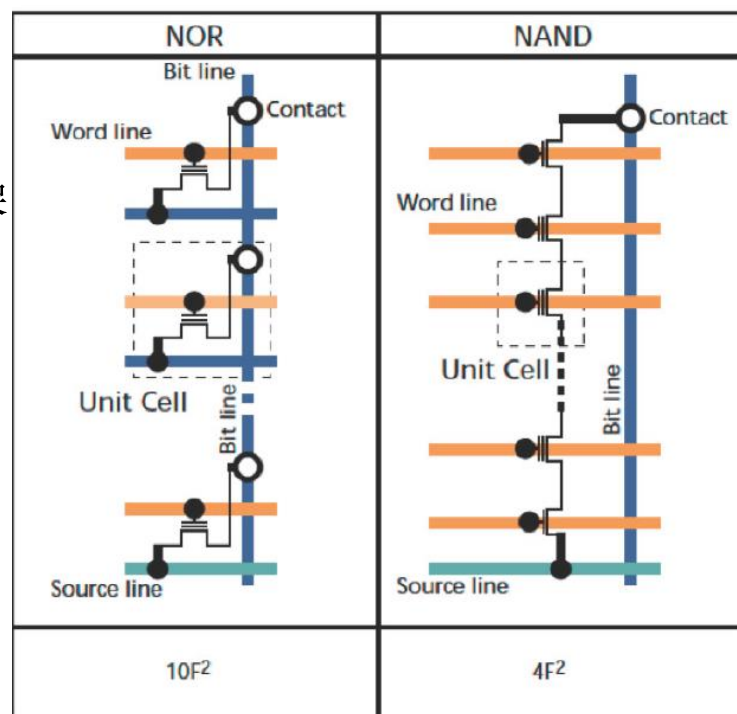
- Flash的分类:

- NOR Flash

- 以bit为基本访问单位;
 - 读操作很快、写操作较慢、擦除操作很慢

- NAND Flash

- NAND Flash是以页(若干KB)为单位进行读写操作, 以块(几百KB或几MB)为单位进行擦除操;
 - NAND Flash的读操作相对NOR Flash慢, 但写和擦除操作比NOR Flash快



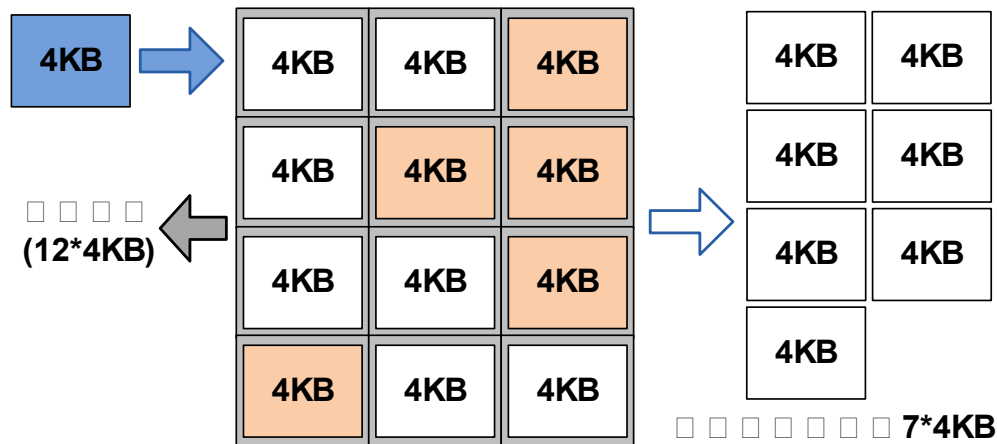
I/O Devices

- Flash的分类:
 - 一个存储单元上存储多个二进制位
 - SLC (Single-Level Cell)
 - SLC的最大可擦除次数一般在10万次
 - MLC (Multi-Level Cell)
 - MLC在5000~10000次
 - TLC (Triple-Level Cell)
 - TLC一般在1000次以内

Device	Read (μ s)	Program (μ s)	Erase (μ s)
SLC	25	200-300	1500-2000
MLC	50	600-900	~3000
TLC	~75	~900-1350	~4500

I/O Devices

- 闪存的写放大现象
 - 根本原因是擦除块>读写单元
 - 擦除块前，先要将其中有效数据搬移到其他未知，即产生了额外的写入量



I/O Devices

- **Flash的优点**

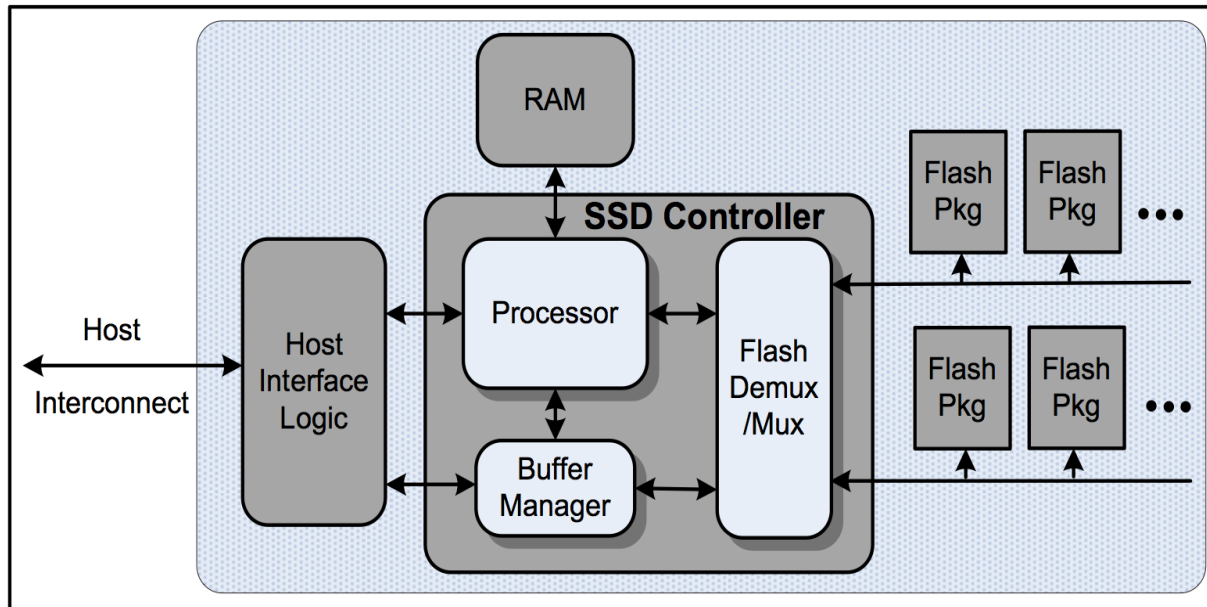
- 高存储密(3D NAND)
- 非易失性(Non-Volatile)
- 节能
- 高性能
- 高可靠性(闪存的MTTF更长；没有机械装置，不受震动等影响)

- **Flash的缺点**

- 价格较高
- 读写不平衡
 - 写本身就比较慢，还有更慢的擦除
- 写入耐久性有限

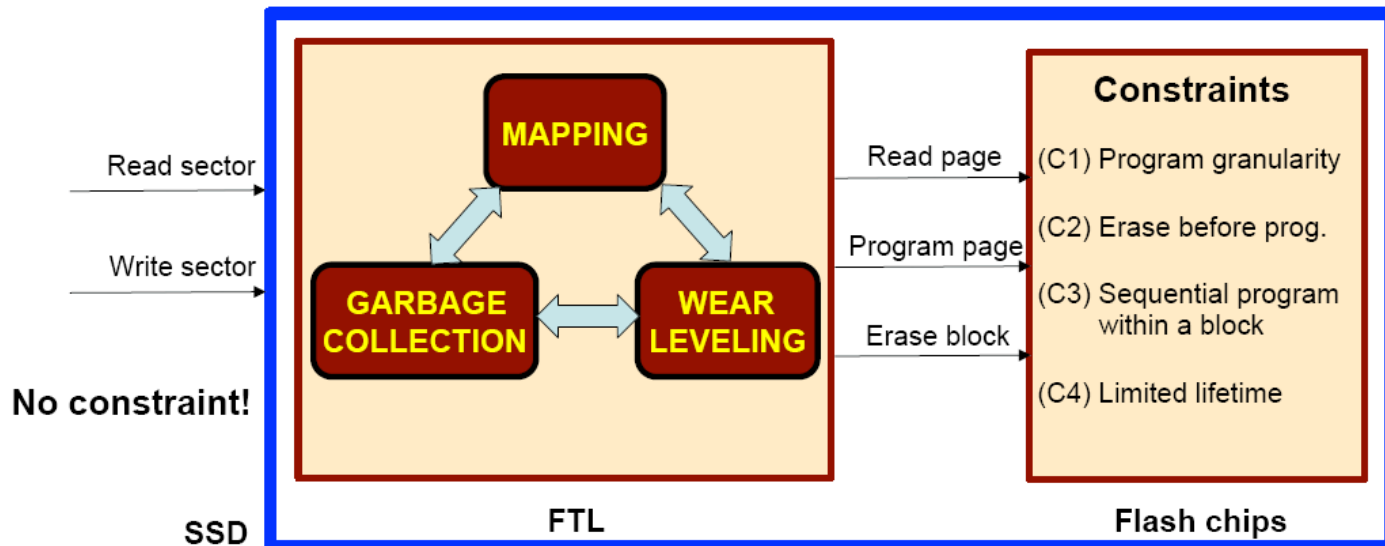
I/O Devices

- 固态硬盘的内部结构
 - 包括接口逻辑模块、处理器、内存、缓冲区管理、多路访问通道和很多闪存芯片
 - 直观的说，固态硬盘本身相当于一台微型的计算机



I/O Devices

- FTL (Flash Translation Layer)
 - 将Flash Chip封装成标准块设备（只有Read, Write两种操作）
 - 地址映射（Mapping）、垃圾回收（Garbage Collection）、磨损平衡（Wear-Leveling）



I/O Devices

- Mapping

- 逻辑页->物理页的映射关系
- **In-memory**, 可以通过Flash上内容**recovery**
- 最简单是**page-level mapping**, 但占用内存空间较大
 - 所以有**block-level mapping**和**hybrid mapping**

Table: 100 → 0 101 → 1 2000 → 2 2001 → 3 Memory

Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a1	a2	b1	b2									
State:	V	V	V	V	i	i	i	i	i	i	i	i	

I/O Devices

- Block-level mapping

- 节省空间
- 但是粒度粗，不太方便
 - 例如只修改c'，也会导致整个block要搬移

Table:	500	→	4	Memory
Block:	0	1	2	
Page:	00 01 02 03	04 05 06 07	08 09 10 11	Flash Chip
Content:	   	a b c d	   	
State:	i i i i	V V V V	i i i i	
Table:	500	→	8	Memory
Block:	0	1	2	
Page:	00 01 02 03	04 05 06 07	08 09 10 11	Flash Chip
Content:	   	   	a b c' d	
State:	i i i i	E E E E	V V V V	

I/O Devices

- Hybrid Mapping
 - 借鉴以上两者的优点，多数采用block-level mapping (data table)，少数log blocks采用page-level mapping，用来记录新更新的pages (log table)
 - 访问流程：
 - 先访问log table;
 - 不在其中的话，再找data table

- Hybrid Mapping

- ## Memory

Flash
Chip

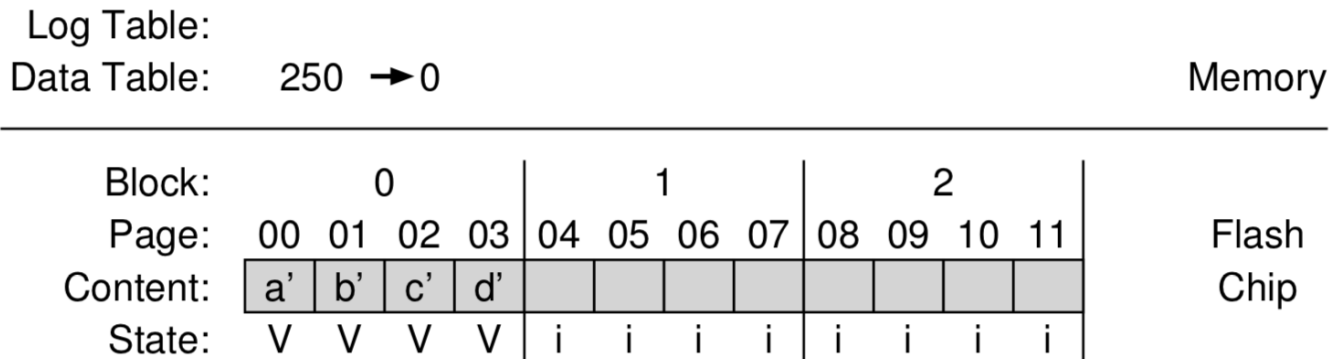
- ## Memory

Flash
Chip

I/O Devices

- Hybrid Mapping

- 为了减少内存开销，log table要尽量通过合并操作，转移到data table中
- 例如以上例子中，block 0中新更新数据和block 2中的原始数据，定期会合并为一个新的block，然后只需要block-level mapping
- 这个例子很幸运，只需要切换block，因此叫switch merge; 开销是擦除一个block



- Hybrid Mapping

- Log Table: 1000 $\rightarrow$ 0 1001 $\rightarrow$ 1
Data Table: 250 $\rightarrow$ 8

Memory

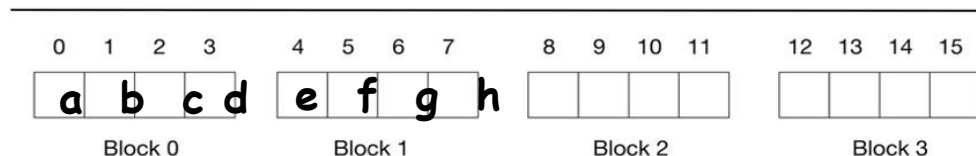
- Flash
-
- Chip

课堂练习

- SSD内部FTL算法采用hybrid mapping的方式进行虚拟地址到物理地址的映射，block table中记录逻辑block id与物理地址的映射关系，log table记录逻辑page id与物理地址的映射关系（ $\text{page id} = 4 * \text{block id}$ ）；
- 同时最多只能有1个log block存在，每个原始block对应一个log block（即一个log block中的数据不能来自于不同的原始block），选择log block进行合并操作时优先选择代价小的进行，选择空白block写入时采用first-fit原则；
- 初始状态如下图所示，到达的请求序列为a, b, c, d, e, e, f, g, e, c，请画出序列结束后的状态，计算发生了几次merge，分别是什么类型？

Log Table:

Block Table: **250-0, 251-4**

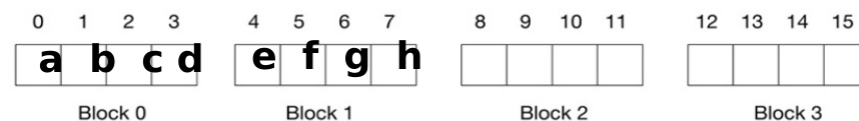


练习答案 (1/3)

- 初始状态如下图所示，到达的请求序列为a, b, c, d, e, e, f, g, e, c，请画出序列结束后的状态

Log Table:

Block Table: 250-0, 251-4

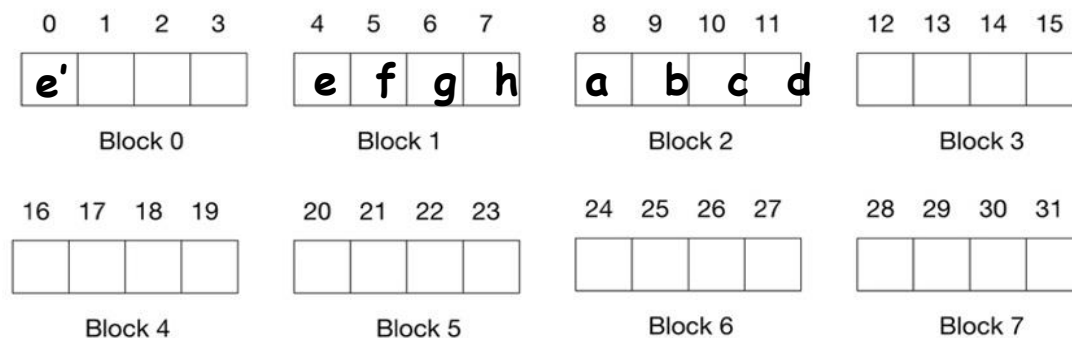


第一个e
到达:

Log Table: 1004-0

Block Table: 250-8, 251-4

Switch
merge

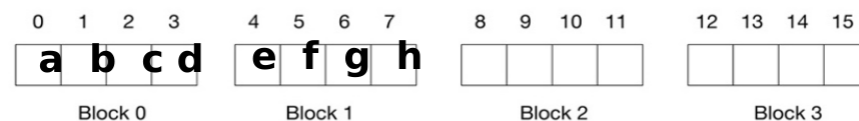


练习答案 (2/3)

- 初始状态如下图所示，到达的请求序列为a, b, c, d, e, e, f, g, e, c，请画出序列结束后的状态

Log Table:

Block Table: 250-0, 251-4

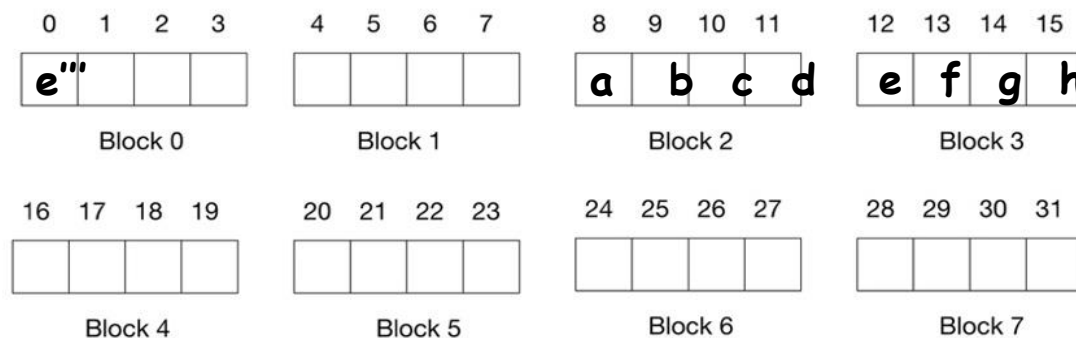


第3个e
到达:

Log Table: 1004-0

Block Table: 250-8, 251-12

Full merge

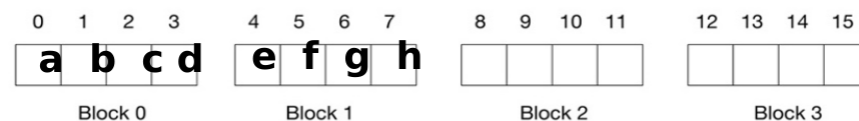


练习答案 (3/3)

- 初始状态如下图所示，到达的请求序列为a, b, c, d, e, e, f, g, e, c，请画出序列结束后的状态

Log Table:

Block Table: 250-0, 251-4

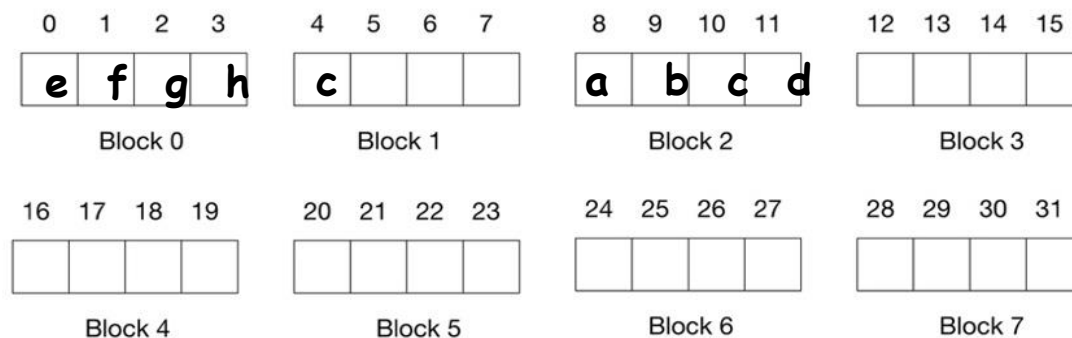


c到达:

Log Table: 1002-4

Block Table: 250-8, 251-0

Partial
merge



I/O Devices

- Garbage Collection

- 回收invalid page占用空间
- 例如a1和a2失效后，可以搬移b1/b2，然后擦除block 0

Table:	100	→	4	101	→	5	2000	→	2	2001	→	3	Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:	a1	a2	b1	b2	c1	c2							
State:	V	V	V	V	V	V	E	E	i	i	i	i	
Table:	100	→	4	101	→	5	2000	→	6	2001	→	7	Memory
Block:	0				1				2				Flash Chip
Page:	00	01	02	03	04	05	06	07	08	09	10	11	
Content:					c1	c2	b1	b2					
State:	E	E	E	E	V	V	V	V	i	i	i	i	

I/O Devices

- **Wear Leveling**
 - 磨损平衡
 - 静态**WL**: 定期交换磨损最严重和最不严重的两个block中的数据
 - 动态**WL**: 实时观测磨损情况, 尽量使用磨损小的block
- **WL**关系到**SSD**的写耐久性, 也和**GC**、**Address Mapping**有着紧密的联系
 - 反映**SSD**写耐久性的两个指标: **Drive Write Per Day (DWPD)**和**Petabyte Write (PBW)**
$$PBW = DWPD * SSD \text{容量} * \text{质保天数}$$

I/O Devices

- 更新一代的SSD
 - Intel Optane
 - 内部采用PCM芯片+大容量缓存
 - 性能和耐久性比Flash-based SSD提升很多
 - 企业级：P4800X、P5800X
 - 消费级：900P



DWPD: 100(P4800X), 10 (900P)

I/O Devices

- 更新一代的SSD
 - 也有基于SLC/MLC Flash优化而来的存储介质：
 - Samsung ZNAND: SZ1735,
 - Kioxia XL-Flash: FL6系列



DWPD: 30-60

I/O Devices

- 叠瓦式硬盘
 - Shingled magnetic recording (SMR)
 - 后续技术: BPMR, HAMR, MAMR

SMR Writes



Figure 2. Track Spacing Enabled by SMR Technology

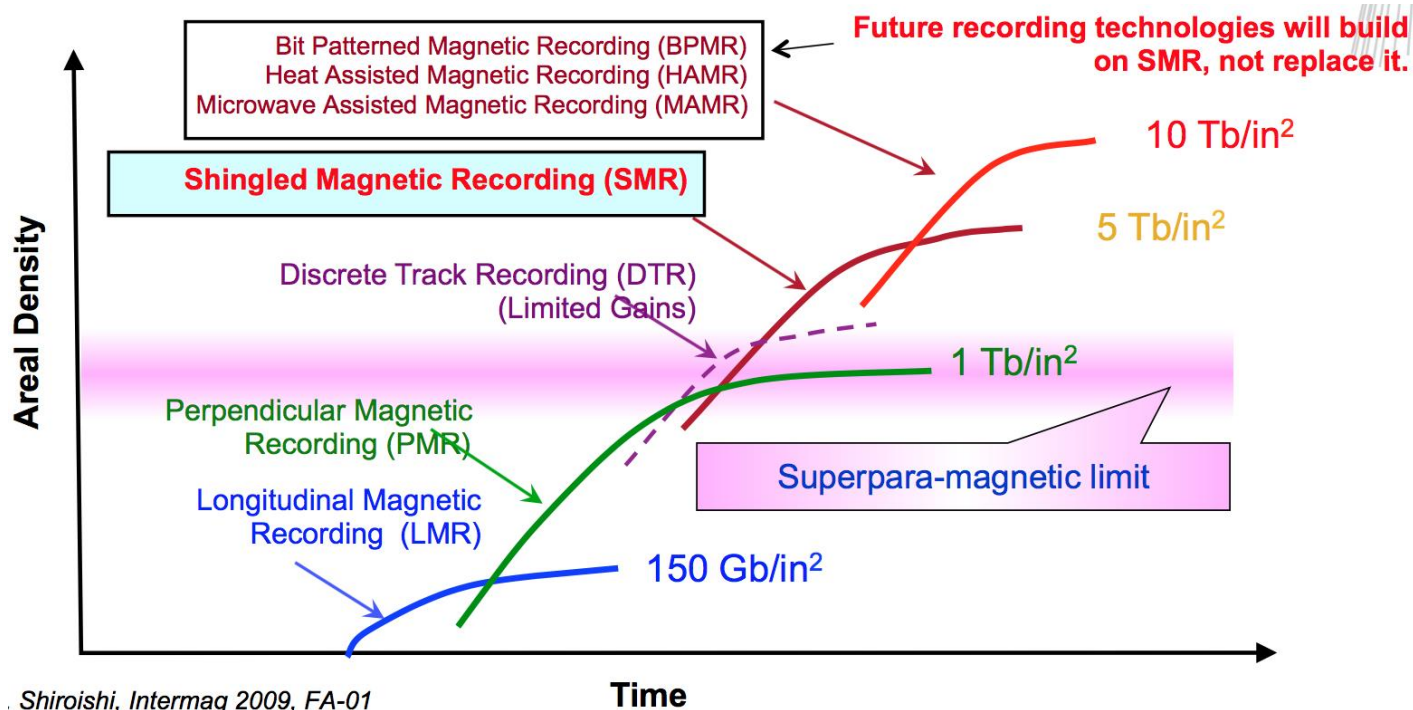


Figure 4. SMR Band Structure

I/O Devices

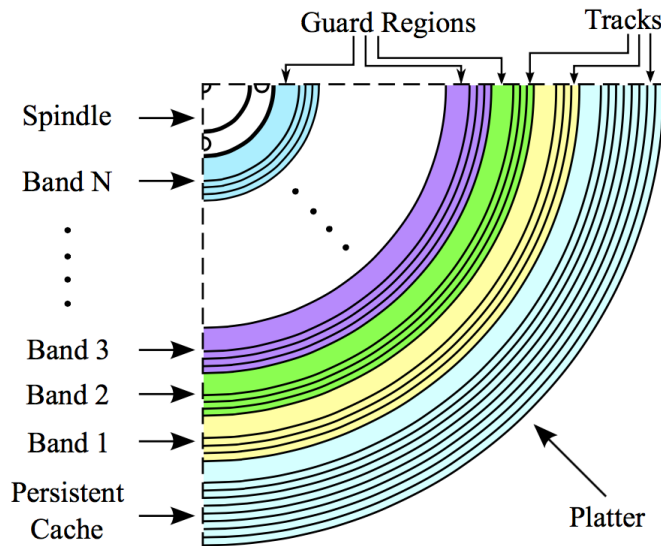
- 叠瓦式硬盘

- 增加存储密度，降低成本，随机写性能较差
- 适用于备份系统、冷存储、科学计算（天文、生物、医学）、日志等

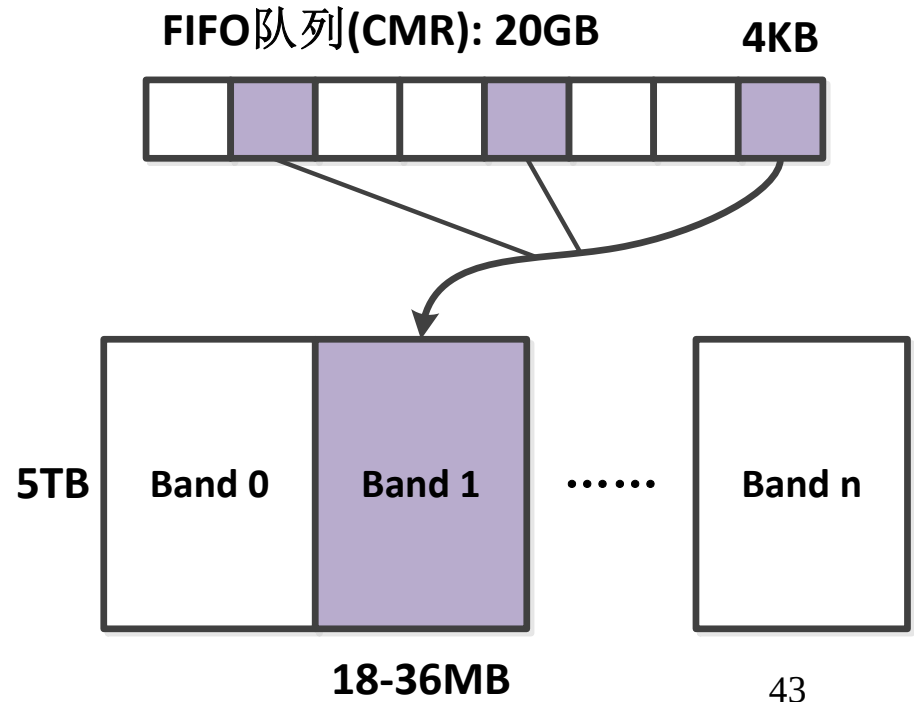


I/O Devices

- SMR分类
 - Driver Managed
 - Host Managed



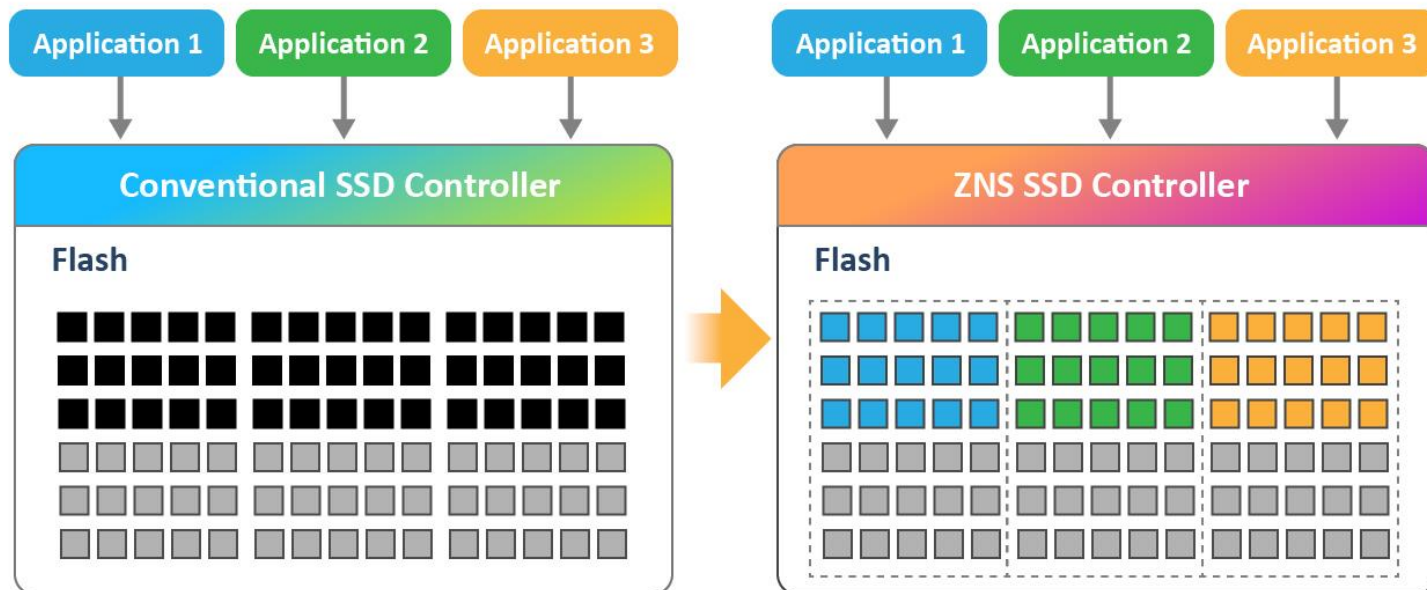
- Seagate 8TB SMR

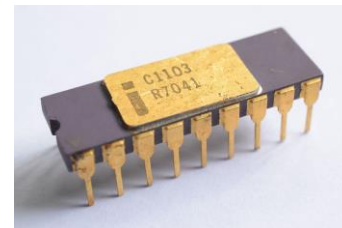


I/O Devices



- ZNS SSD
 - Zoned Namespace SSD
 - 将 **FTL** 的部分功能（主要是**GC**）上移到应用中，可能减少开销





国产半导体存储的发展 - DRAM

- 1966年，IBM工程师Robert H. Dennard发明DRAM
- 1970年，Intel发布第一款商业化DRAM C1103（1Kb）
- 1975年，北大物理系与109厂1Kb DRAM
- 1981年，中科院半导体所16Kb DRAM
- 1985年，无锡江南无线电器材厂64Kb DRAM
- 1990年前后，日本DRAM产业被迫向韩国和美国转移，三星、美光等企业在半导体存储方面开始领先
- 1993年，无锡华晶256Kb DRAM，此前一年三星已经发布16Mb DRAM
- 1999年，上海华虹引进日本NEC技术生产64Mb DRAM，2004年转型晶圆代工

国产半导体存储的发展 - DRAM

- 2004年，中芯国际开始投产**DRAM**，后为奇梦达(Qimonda，原西门子**DRAM**部门)等厂商代工**DRAM**
- 2008年，中芯国际收缩业务，放弃**DRAM**制造
- 国产**DRAM**开始长达8年的空白期。。。
- 2010s中，智能手机和数据中心的发展使**DRAM**需求剧增
- 2016年，福建晋华与台湾联电签订协议，由联电代为研发**DRAM**技术，联电从美光(Micron)挖了部分技术人员
 - 2017年12月，美光起诉晋华和联电，晋华和联电反诉
 - 2018年10月，晋华被美国商务部禁运
 - 2023年12月，晋华与美光和解，但已错过了关键的市场机遇

国产半导体存储的发展 - DRAM

- 奇梦达在**2008**金融危机前是世界第四大**DRAM**厂商，”欧洲半导体存储之光”
 - **2009**年**4**月破产，华芯、浪潮、紫光先后等接手了奇梦达在西安和欧洲的部分研发中心和产线（今天的紫光**DRAM**业务）
 - **2015**年**6**月，奇梦达核心专利被英飞凌打包出售
- **2016**年，合肥长鑫成立，获得了奇梦达**DRAM**相关的核心技术
- **2019**年，长鑫在奇梦达**46nm DRAM**基础上，实现了**10nm DRAM**技术，并继续收购了奇梦达其余专利
- **2023**年，长鑫发布**12Gb LPDDR5 DRAM**颗粒

国产半导体存储的发展 - NAND Flash

- 1986年，东芝工程师舂冈富士雄博士发明NAND Flash
- 1991年，东芝发布4MB NAND
- 1992年，Intel发布12MB NAND
- 1999年，三星发布1GB NAND
 - 2002年朗科发明U盘
- 2007年，东芝发布3D NAND技术BiCS
- 2013年，三星发布3D NAND技术V-NAND
- 2016年，Intel和美光发布3D NAND技术CuA
- 2016年，武汉长江存储成立，进入NAND行业
- 2018年，长江存储发布3D NAND技术XTacking，以及一款32层国产NAND
 - 此时国际领先水平是96层

国产半导体存储的发展 - NAND Flash

- 2019年9月，长江存储发布基于XTacking的64层NAND
 - 此时国际领先水平是128层
- 2020年4月，长江存储试产128层NAND，并于2021年9月量产（此时国际领先水平是192层）
- 2022年8月，长江存储试产232层NAND，领先或追平三星、Intel/美光、kioxia（原东芝NAND部门）
- 2022年12月，TechInsights在一款海康威视的SSD中拆解确认了长江存储232层XTacking NAND已经商用
 - TechInsights称这是其确认的世界上首款商用200+层3D NAND
 - 2022年12月底，美国商务部将长江存储加入实体清单
 - 2023年3月，国家半导体大基金宣布对长江存储增加129亿投资

技术发展的根本动力是市场需求

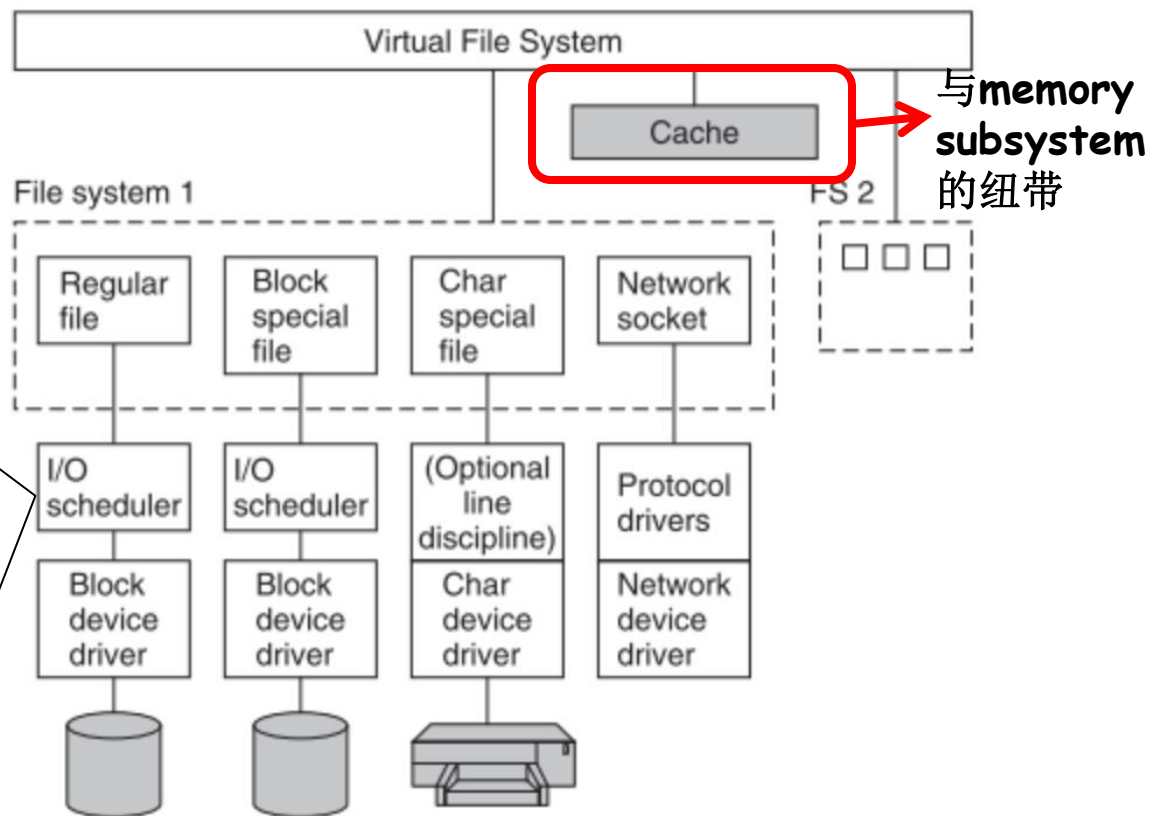
Outline

- I/O Devices
- **Unix I/O**
- Reading File Metadata
- Sharing Files & I/O redirection

The I/O subsystem in Unix/Linux-like OS

Unix I/O

Linux中的I/O scheduler负责对FS中产生的read/write requests进行reordering和merging, 以协助device提高I/O效率, 常用的包括:
noop
cfq (completely fair queuing)
deadline



Why Unix I/O

- Input/Output
 - A process of copying data between main memory and external devices
- Standard I/O library
 - User level or high level I/O functions
- Unix I/O
 - I/O functions provided by kernel
 - Helps you understand other system concepts
 - I/O vs. process
 - Sometimes, it is the only choice

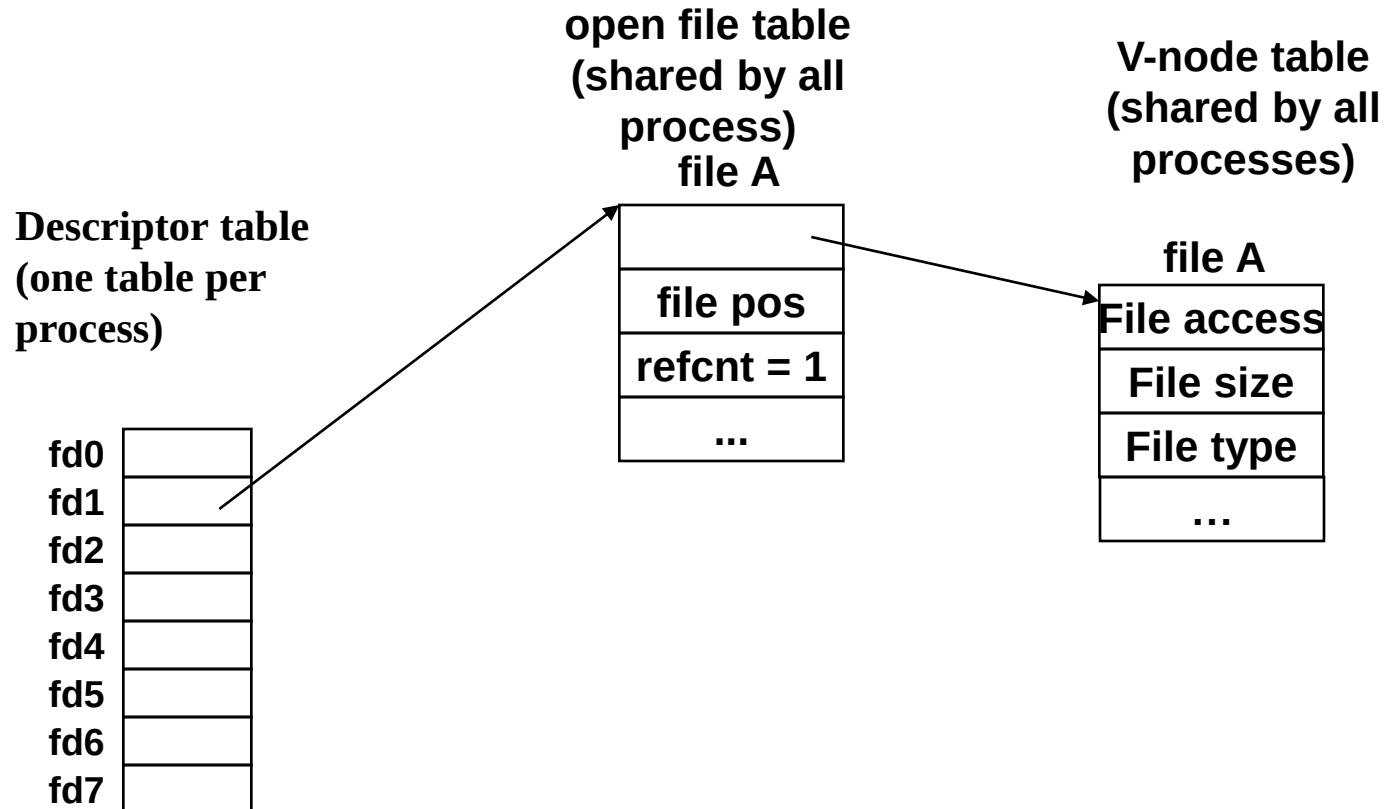
Unix I/O

- A Unix file is a sequence of m bytes
 - $B_0, B_1, \dots, B_k, \dots, B_m$
- All I/O devices are **modeled as files**
 - such as networks, disks, and terminals
 - allows Unix to export a simple, low-level application interface, known as Unix I/O
- All input and output is performed by reading and writing the appropriate files
 - Enables all input and output to be performed in a uniform and consistent way.

Open Files

- An application announces its intention to access an I/O device
 - The kernel opens the corresponding file
 - The kernel returns a **small non-negative integer, descriptor**
 - Identifies the file in all subsequent operations on the file
 - The kernel keeps track of all information about the open file
 - Maintains a file position k , initially 0, for each open file

Kernel Data Structures for Files



Descriptor table

- 每个进程有自己独立的**descriptor table**
 - 每个文件描述符是一个非负数字，作为**descriptor table**的索引
 - 0：标准输入（默认已打开）
 - 1：标准输出（默认已打开）
 - 2：标准错误（默认已打开）
 - 其他打开的文件，从**3**开始编号
 - 每个打开的文件描述符指向一个**file table**中的一个**entry**

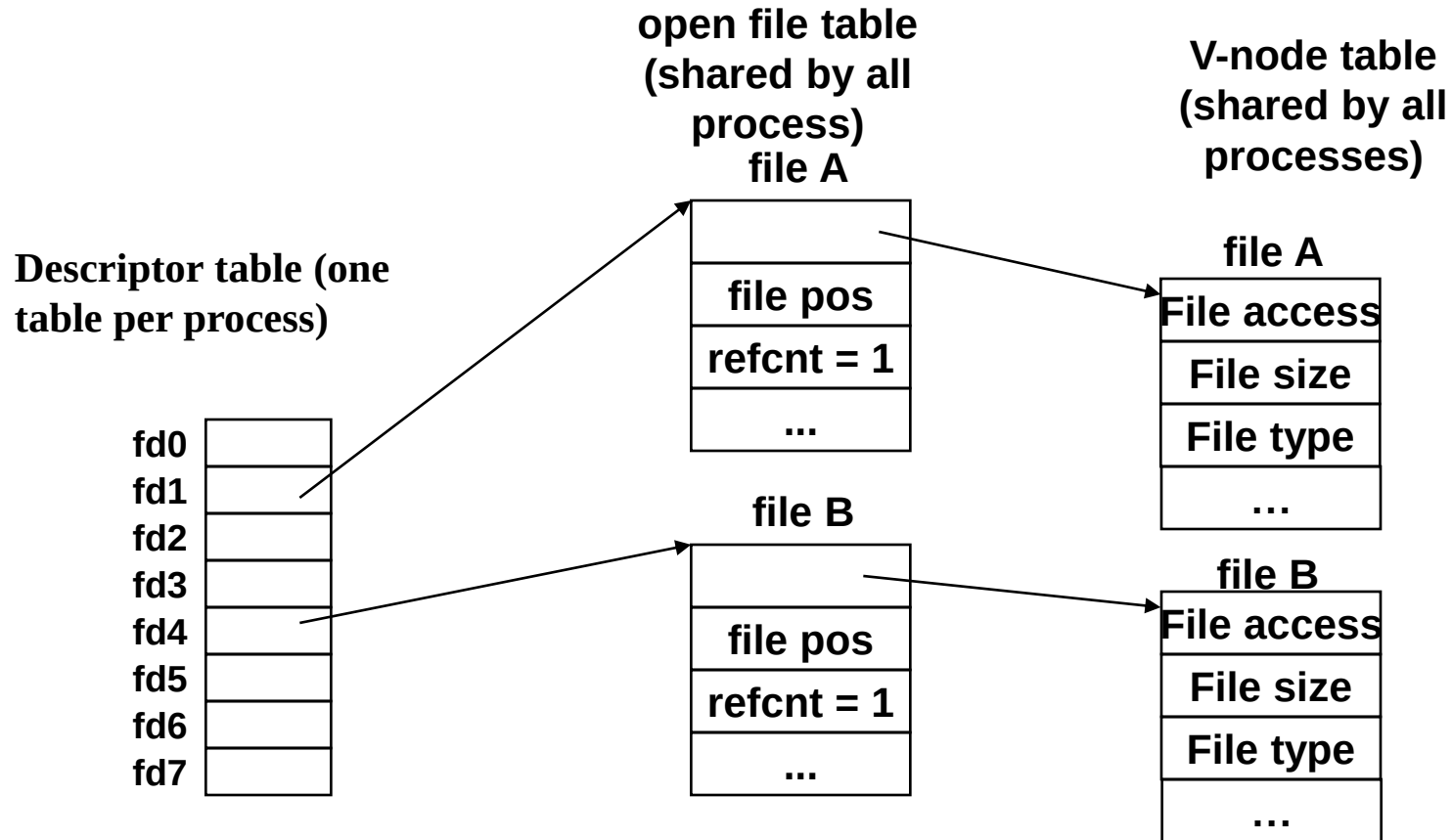
File table

- The set of **open files** is represented by a file table
- File table is **shared** by all processes
- Each file table entry consists
 - the **current file position**
 - a **reference count** of the number of descriptor entries that currently point to it
 - a **pointer** to an entry in the v-node table
 - the kernel deletes a file table entry when its reference count becomes **zero**

V-node table

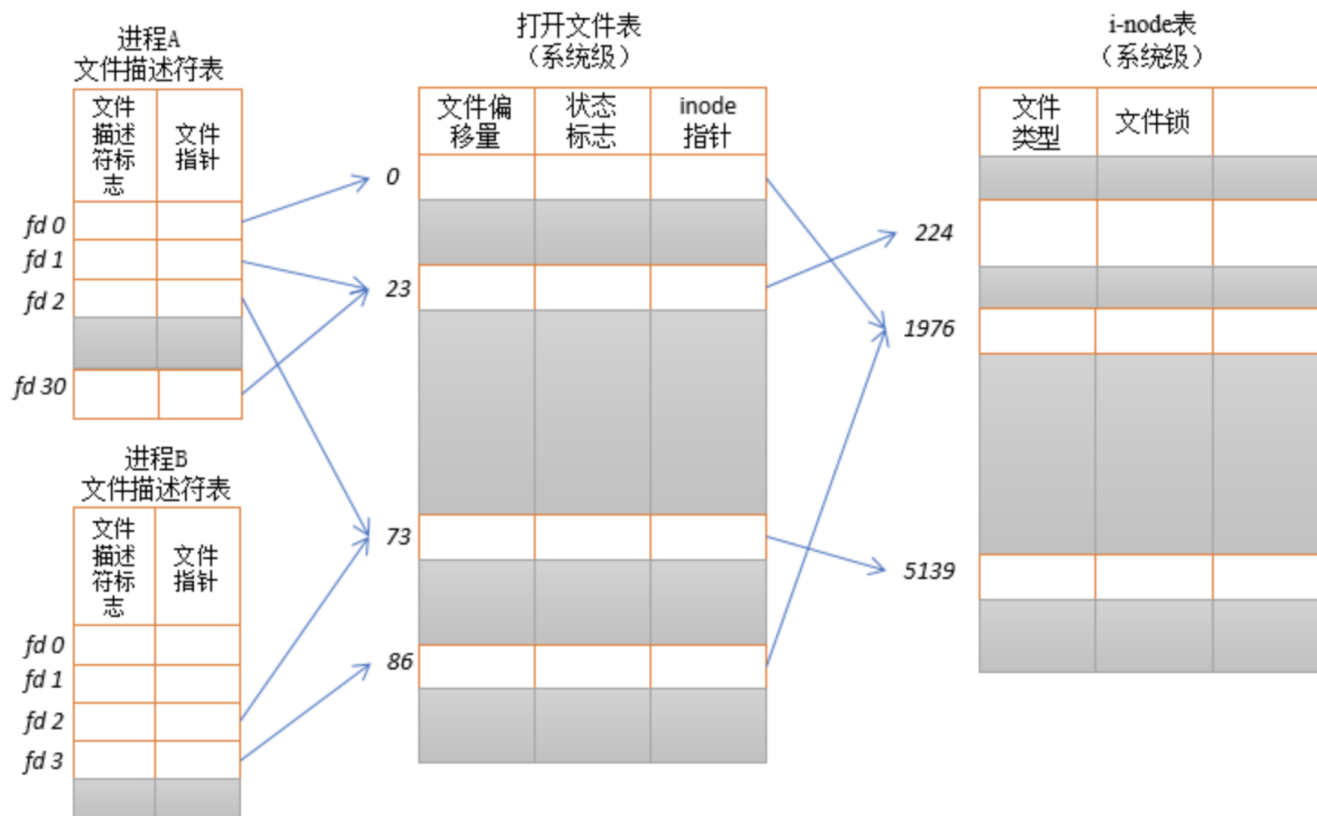
- V-node table is shared by all processes
- Each entry contains most of the information of a file

Kernel Data Structures for Files



例子

- 可能一个进程多个**fd**对应打开同一个打开的文件(如**dup2**)
- 可能多个进程对应同一个打开文件(如**fork**)
- 可能多个进程对应不同打开文件，但是同一个文件



Open Files

- An application announces its intention to access an I/O device
 - 应用程序只需要维护文件描述符，其他信息都在kernel数据结构中
 - 一个应用程序可以通过seek操作，显式的设置文件的当前访问位置
 - Read/write操作都从当前位置开始向后进行

Open Files

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open(char *filename, int flags, mode_t mode);
```

Returns: new file descriptor if OK, -1 on error

flags

- O_RDONLY, O_WRONLY, O_RDWR (must have one)
- O_CREAT, O_TRUNC, O_APPEND, O_DIRECT (optional)

mode

- S_IRUSR, S_IWUSR, S_IXUSR
- S_IRGRP, S_IWGRP, S_IXGRP
- S_IROTH, S_IWOTH, S_IXOTH

Open Files

- **Flags:**
 - **O_CREAT**: 如果文件不存在, 就创建一个空文件
 - **O_TRUNC**: 如果文件已经存在, 就截断它
 - 再写入时, 从头写入, 覆盖原来的内容
 - **O_APPEND**: 在每次写入操作前, 设置文件位置到文件的结尾处
 - **O_DIRECT**: 跳过 (by pass) `cache` 直接以读写文件
- **Note**: `open` 集成了 `create file` 功能

Open Files

- Mode:

掩码	描述
S_IRUSR S_IWUSR S_IXUSR	使用者（拥有者）能够读这个文件 使用者（拥有者）能够写这个文件 使用者（拥有者）能够执行这个文件
S_IRGRP S_IWGRP S_IXGRP	拥有者所在组的成员能够读这个文件 拥有者所在组的成员能够写这个文件 拥有者所在组的成员能够执行这个文件
S_IROTH S_IWOTH S_IXOTH	其他人（任何人）能够读这个文件 其他人（任何人）能够写这个文件 其他人（任何人）能够执行这个文件

Open Files

umask() : set mask of process

利用**umask**命令可以指定哪些权限将在新文件的默认权限中被删除

```
#define DEF_MODE S_IRUSR | S_IWUSR |      \  
                S_IRGRP | S_IWGRP |      \  
                S_IROTH | S_IWOTH  
#define DEF_UMASK S_IWGRP | S_IWOTH
```

```
umask(DEF_UMASK);
```

```
fd = open ("foot.txt",  
           O_CREAT|O_TRUNC|O_WRONLY, DEF_MODE);
```

```
/* permission: S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH*/
```

Open Files

- 一个进程的文件描述符有上限
 - 超过了会报错: **"Too many open files"**
 - 单个进程的上限一般是**1024**
 - 可以通过**ulimit -n**查询
 - 系统的最大值: **sysctl -a | grep fs.file-max**
 - 及时关闭不再被使用的文件描述符

Close Files

- close the file
 - Do not access the file
 - The kernel action
 - The kernel frees the structures it created when the file was opened
 - The kernel restores the descriptor to a pool of available descriptors
 - The next file that is opened is guaranteed to receive the smallest available descriptor in the pool

Close Files

- close the file
 - Default actions when terminate
 - The kernel closes all open files
 - The kernel frees their memory resources

```
#include <unistd.h>  
int close(int fd) ;
```

Returns: zero if OK, -1 on error

课堂练习

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6
7      fd1 = Open("foo.txt", O_RDONLY, 0);
8      Close(fd1);
9      fd2 = Open("baz.txt", O_RDONLY, 0);
10     printf("fd2 = %d\n", fd2);
11     exit(0);
12 }
```

屏幕输出什么？

Reading and Writing Files

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

returns: number of bytes read if OK,
0 on EOF, -1 on error

```
ssize_t write(int fd, const void *buf, size_t count);
```

returns: number of bytes written if OK,
-1 on error

Reading and Writing Files

- A read operation
 - Copies $m > 0$ bytes from the file to memory
 - starting at the **current** file position k
 - incrementing k by m
 - If the total bytes from k to the end of file is less than m , it triggers a condition known as end-of-file (EOF)
 - Can be detected by the application
 - There is **no explicit** "EOF character" at the end of a file

Reading and Writing Files

```
1  #include "csapp.h"
2
3  int main(void)
4  {
5      char c;
6
7      /* copy stdin to stdout, one byte at a time */
8      while(Read(STDIN_FILENO, &c, 1) != 0)
9          Write(STDOUT_FILENO, &c, 1);
10     exit(0);
11 }
```

STDIN_FILENO(0), STDOUT_FILENO(1), STDERR_FILENO(2)

Reading and Writing Files

- `ssize_t` vs. `size_t`
 - X86-64:
 - `ssize_t`: signed long
 - `size_t`: unsigned long
- Short count
 - Read and write transfer **fewer** bytes than the application requests
 - Encounter EOF on reads
 - Reading text lines from a terminal
 - Reading and writing network sockets

Outline

- I/O Devices
- Unix I/O
- **Reading File Metadata**
- Sharing Files & I/O redirection

Reading File Metadata

```
#include <unistd.h>
```

```
#include <sys/stat.h>
```

```
int stat(const char *filename, struct stat *buf);
```

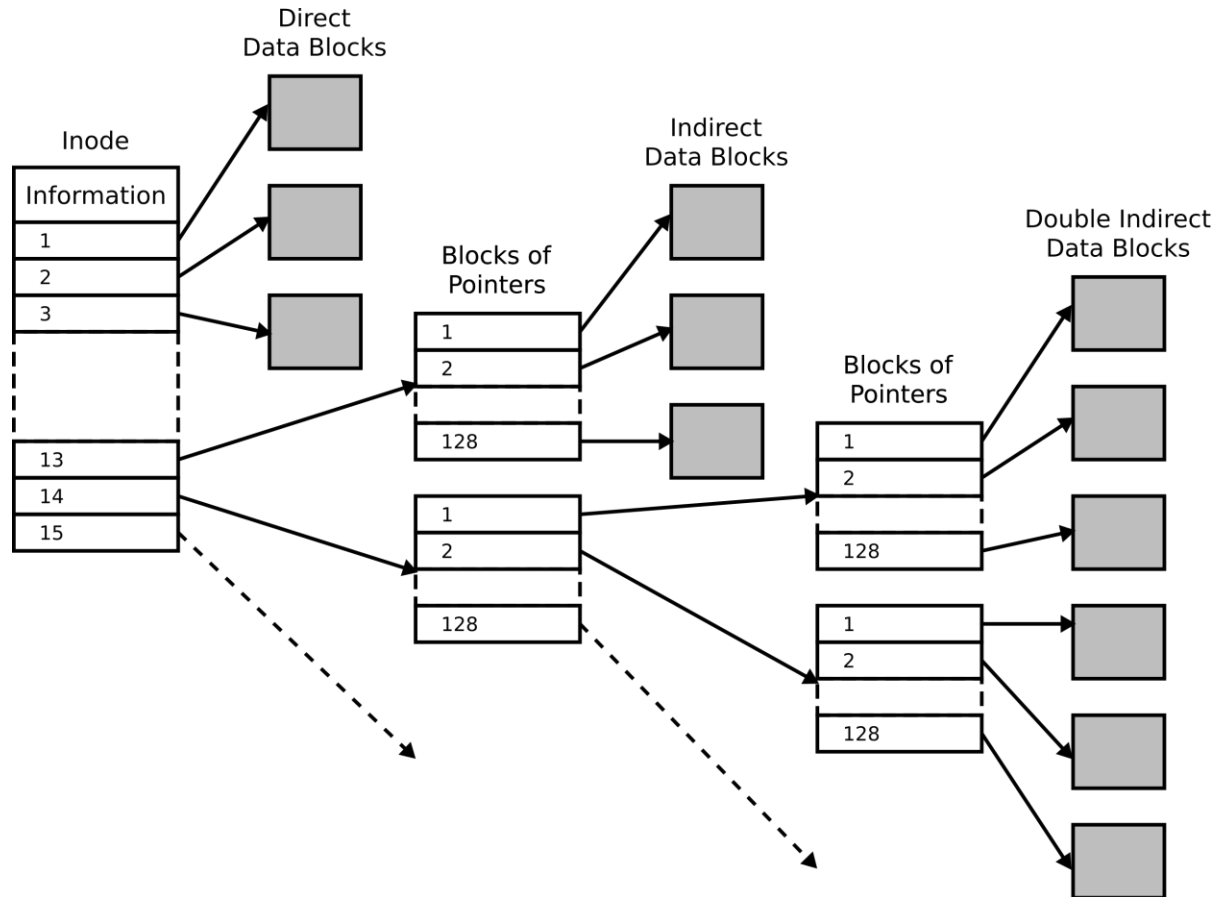
```
int fstat(int fd, struct stat *buf) ;
```

returns: 0 if OK, -1 on error

Reading File Metadata

```
/* file info returned by the stat function */
struct stat {
    dev_t st_dev;           /* device */
    ino_t st_ino;           /* inode number */
    mode_t st_mode;         /* protection and file type */
    nlink_t st_nlink;       /* number of hard links */
    uid_t st_uid;           /* user ID of owner */
    gid_t st_gid;           /* group ID of owner */
    dev_t st_rdev;          /* device type (if inode device) */
    off_t st_size;          /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks; /* number of blocks allocated */
    time_t st_atime;        /* time of last access */
    time_t st_mtime;        /* time of last modification */
    time_t st_ctime;        /* time of last change */
};
```

inode and vnode



inode由文件系统保存在外存设备上，打开文件时，**inode**加载到内存中，构建对应的**vnode**在Linux系统中，**vnode**通常称为**generic inode**(定义为**struct inode**)

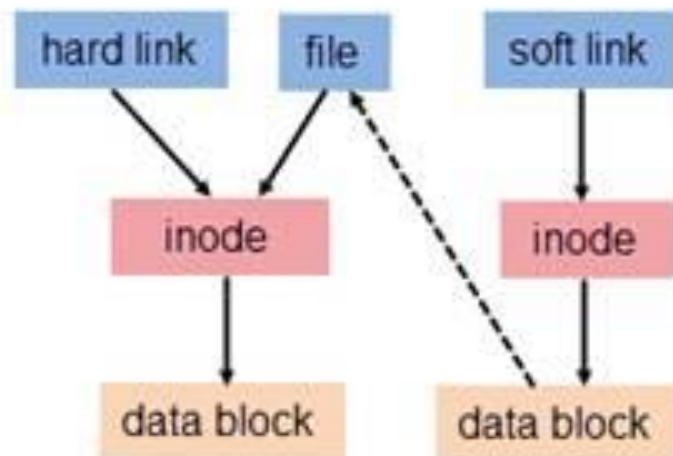
硬链接和软链接

- 解决文件共享使用问题
- 硬链接 (hard link)
 - 一个 **inode** 号对应多个文件名
 - 同一个文件使用了多个别名
 - 硬链接可由命令 **link** 或 **ln** 创建
 - 文件有相同的 **inode** 及 **data block**;
 - 只能对已存在的文件进行创建;
 - 不能交叉文件系统进行硬链接的创建;
 - 不能对目录进行创建, 只可对文件创建;
 - 删除一个硬链接文件并不影响其他有相同 **inode** 号的文件

硬链接和软链接

- 软链接

- 又称符号链接，即 **soft link** 或 **symbolic link**
- 文件用户数据块中存放的内容是另一文件的路径名的指向，则该文件就是软连接
- 软链接就是一个普通文件，只是数据块内容有点特殊。软链接有着自己的 **inode** 号以及用户数据块



硬链接和软链接

```
1 $ ln myfile hard
2 $ ls -li
3
4 25869085 -rw-r--r-- 2 unixzii staff 27 7 8 17:39 hard
5 25869085 -rw-r--r-- 2 unixzii staff 27 7 8 17:39 myfile
```

```
1 $ ln -s myfile soft
2 $ ls -li
3
4 25869085 -rw-r--r-- 2 unixzii staff 36 7 8 17:45 hard
5 25869085 -rw-r--r-- 2 unixzii staff 36 7 8 17:45 myfile
6 25869216 lrwxr-xr-x 1 unixzii staff 6 7 8 17:47 soft -> myfile
```

```

int main(int argc, char **argv)
{
    struct stat stat ;
    char *type, *readok ;
    Stat(argv[1], &stat) ;
    if (S_ISREG(stat.st_mode)) /* Determine file type */
        type = "regular" ;
    else if (S_ISDIR(stat.st_mode))
        type = "directory" ;
    else
        type = "other" ;

    if ((stat.mode & S_IRUSR)) /* check read access */
        readok = "yes";
    else
        readok = "no";

    printf("type: %s, read: %s\n", type, readok) ;
    exit(0) ;
}

```

S_ISREG() Is this a regular file?
 S_ISDIR() Is this a directory file?

Reading Directory

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
DIR *opendir (const char *name);
```

returns: 成功则返回处理的指针；若出错，则返回NULL

```
struct dirent *readdir (DIR *dirp);
```

returns: 成功则返回指向下一个目录项的指针；没有更多目录或出错则返回NULL

Reading Directory

- 目录项

```
struct dirent {  
    ino_t d_ino;          /* inode number */  
    char d_name[256]; /* filename */  
}
```

有些linux版本包括更多内容

Reading Directory

```
#include <dirent.h>
```

```
DIR *closedir (DIR *dirp);
```

returns: 成功则返回0；若出错，则返回-1

Reading Directory

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      DIR *stream;
6      struct dirent *dep;
7
8      stream = Opendir(argv[1]);
9
10     errno = 0;
11     while ((dep = readdir(stream)) != NULL) {
12         printf("Found file: %s\n", dep->d_name);
13     }
14     if (errno != 0)
15         unix_error("readdir error");
16
17     Closedir(stream);
18     exit(0);
19 }
```

Outline

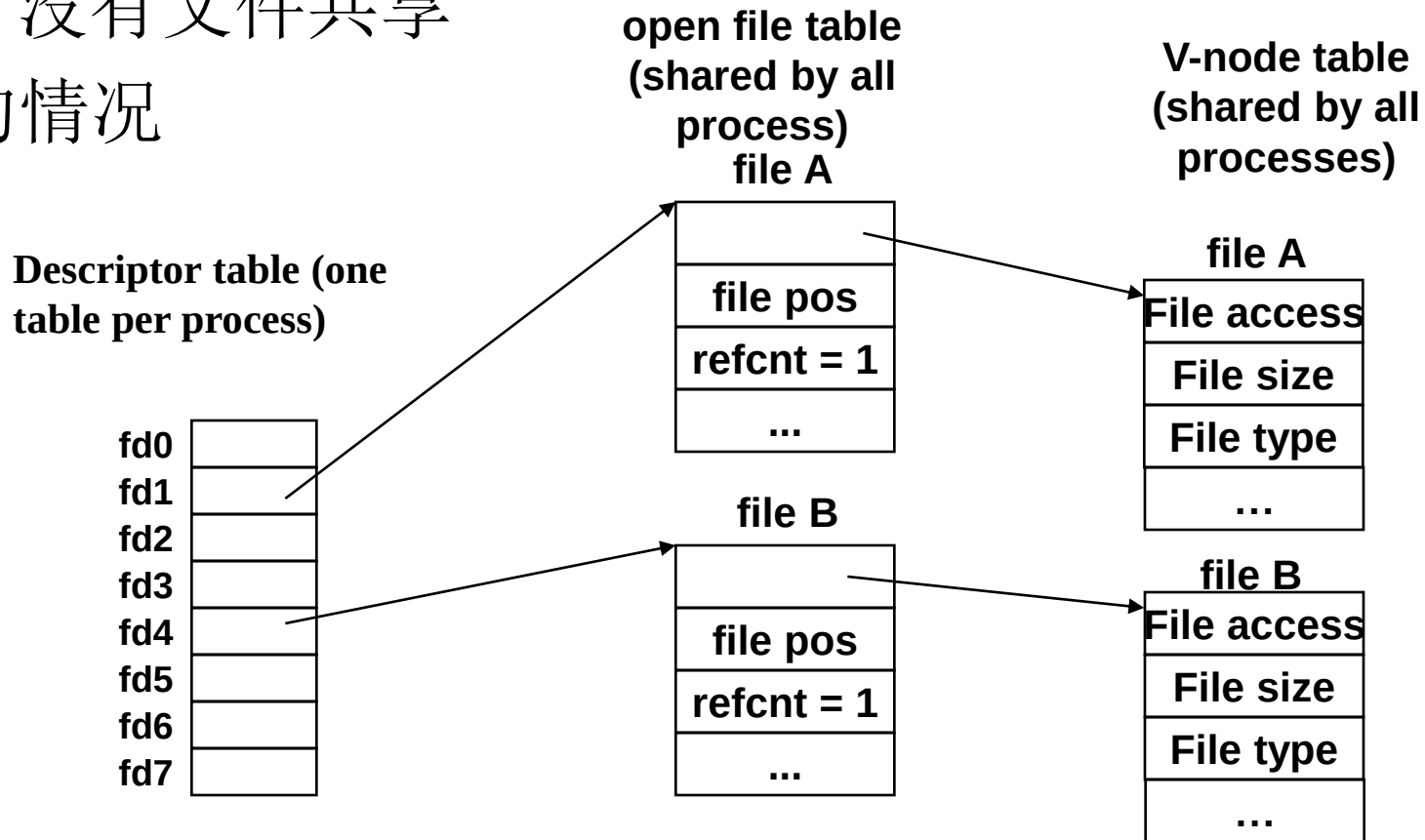
- I/O Devices
- Unix I/O
- Reading File Metadata
- **Sharing Files & I/O redirection**

V-Node table

- Shared by all processes
- Each entry contains most of the information in the stat structure, such as
 - st_mode
 - st_size

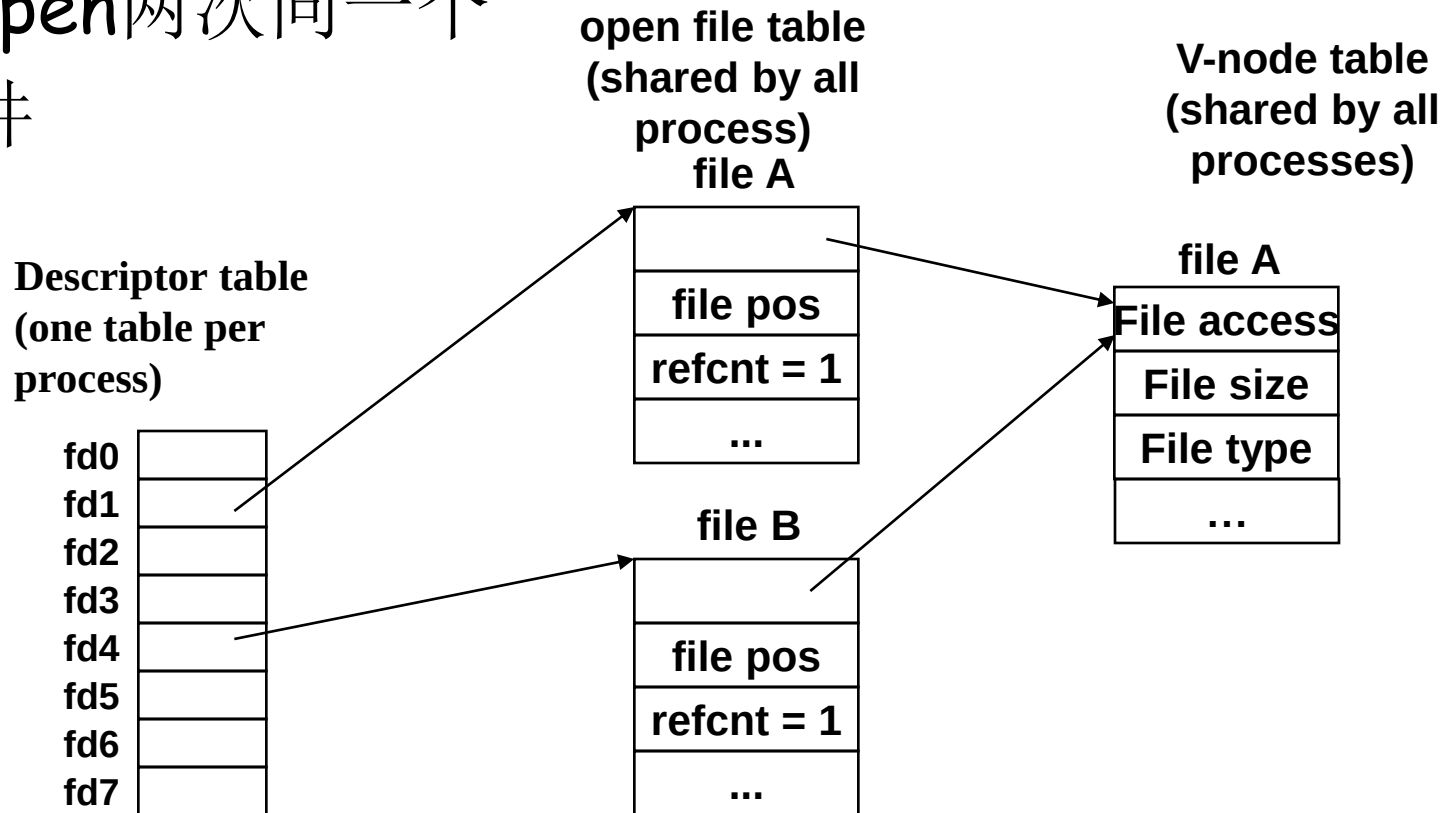
Sharing Files

- 没有文件共享的情况



Sharing Files

- **open**两次同一个文件



Sharing Files

```
#include "csapp.h"
```

```
int main()
```

```
{
```

```
    int fd1, fd2;
```

```
    char c;
```

```
    fd1 = open("foobar.txt", O_RDONLY, 0);
```

```
    fd2 = open("foobar.txt", O_RDONLY, 0);
```

```
    read(fd1, &c, 1);
```

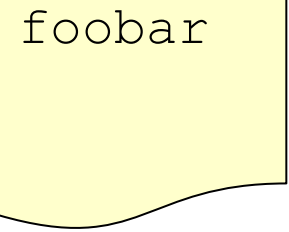
```
    read(fd2, &c, 1);
```

```
    printf("c = %c\n", c);
```

```
    exit(0)
```

```
}
```

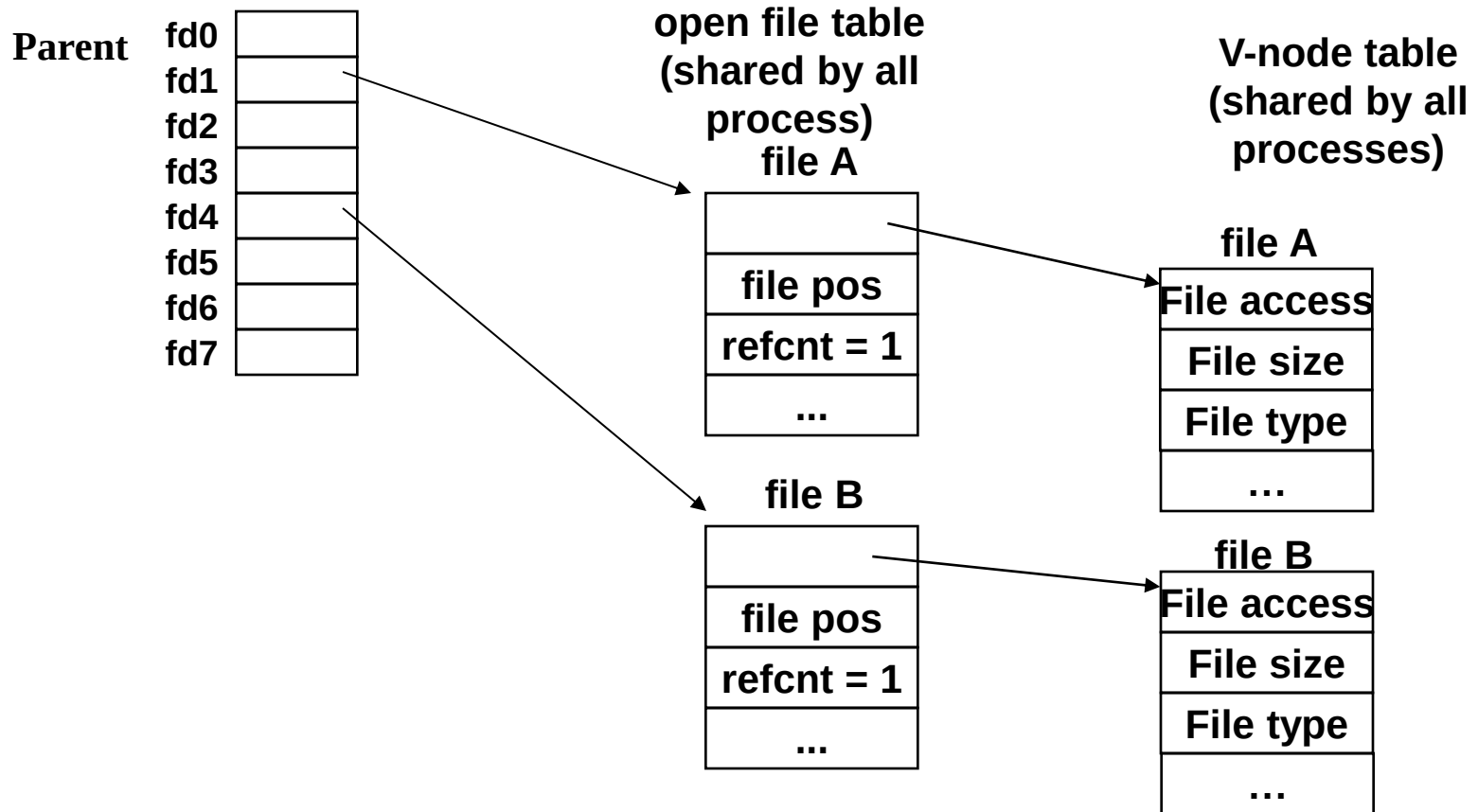
foobar.txt



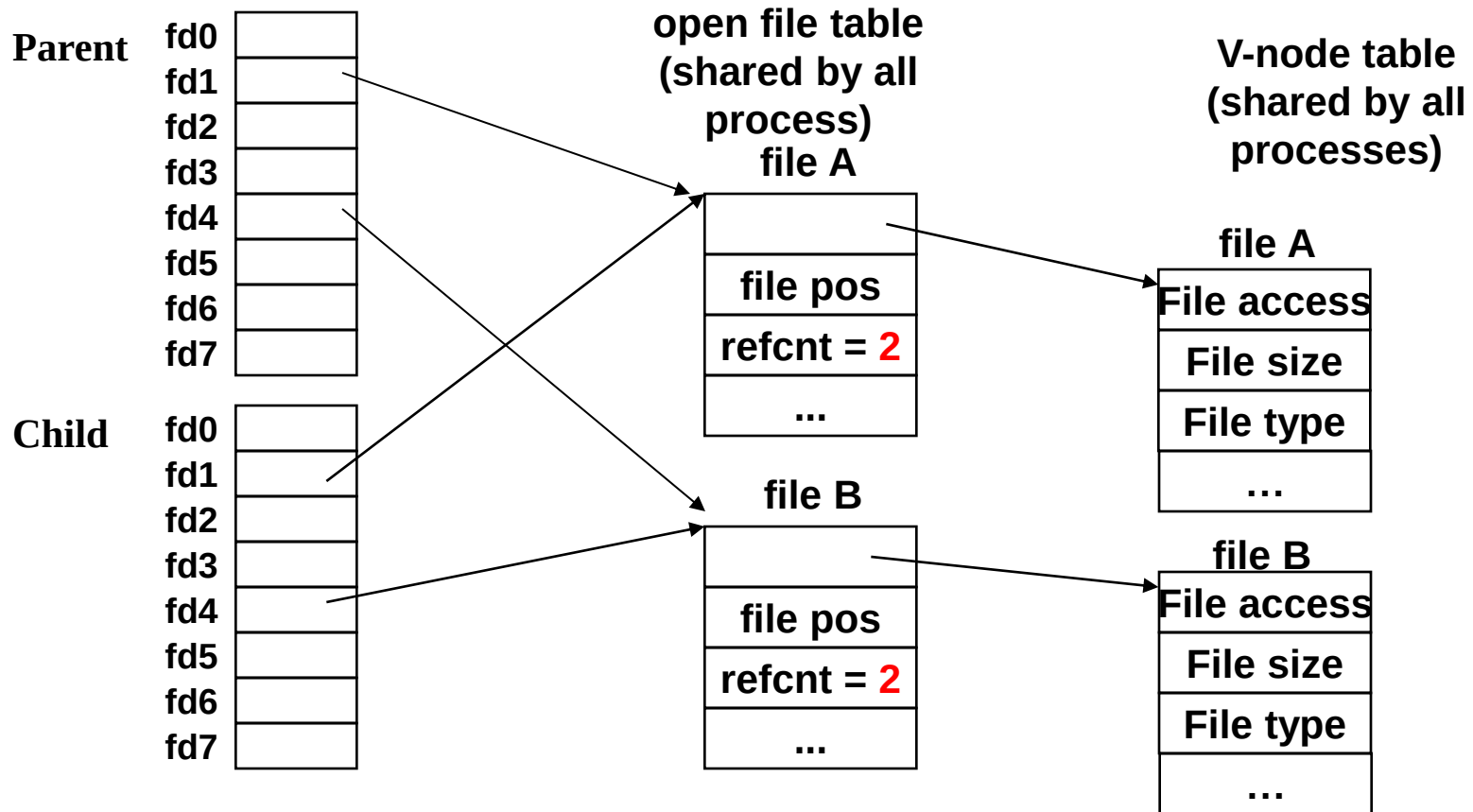
foobar

屏幕输出什么？

Sharing Files between Parent and Child



Sharing Files between Parent and Child



Sharing Files between Parent and Child

```
#include "csapp.h"
```

```
int main()  
{
```

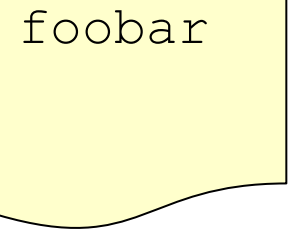
```
    int fd;  
    char c;
```

```
    fd = open("foobar.txt", O_RDONLY, 0) ;  
    if (fork() == 0) {  
        read(fd, &c, 1) ;  
        exit(0) ;  
    }
```

```
    wait(NULL) ;  
    read(fd, &c, 1) ;  
    printf("c = %c\n", c) ;  
    exit(0)
```

```
}
```

foobar.txt



foobar

屏幕输出什么?
为什么? COW?

I/O Redirection

- `unix > ls > foo.txt`
 - 重定向操作，`ls`本来应该输出到屏幕（标准输出，1）
 - 但>将其改为`foo.txt`
 - 如何实现？
 - 将`fd=1`的指针修改，指向打开的`foo.txt`文件
 - `Dup2`操作
- `Ps.` 与`ls | grep xxx`的区别

I/O Redirection

- `dup2` copies entries in the per-process file descriptor table.

```
#include <unistd.h>
```

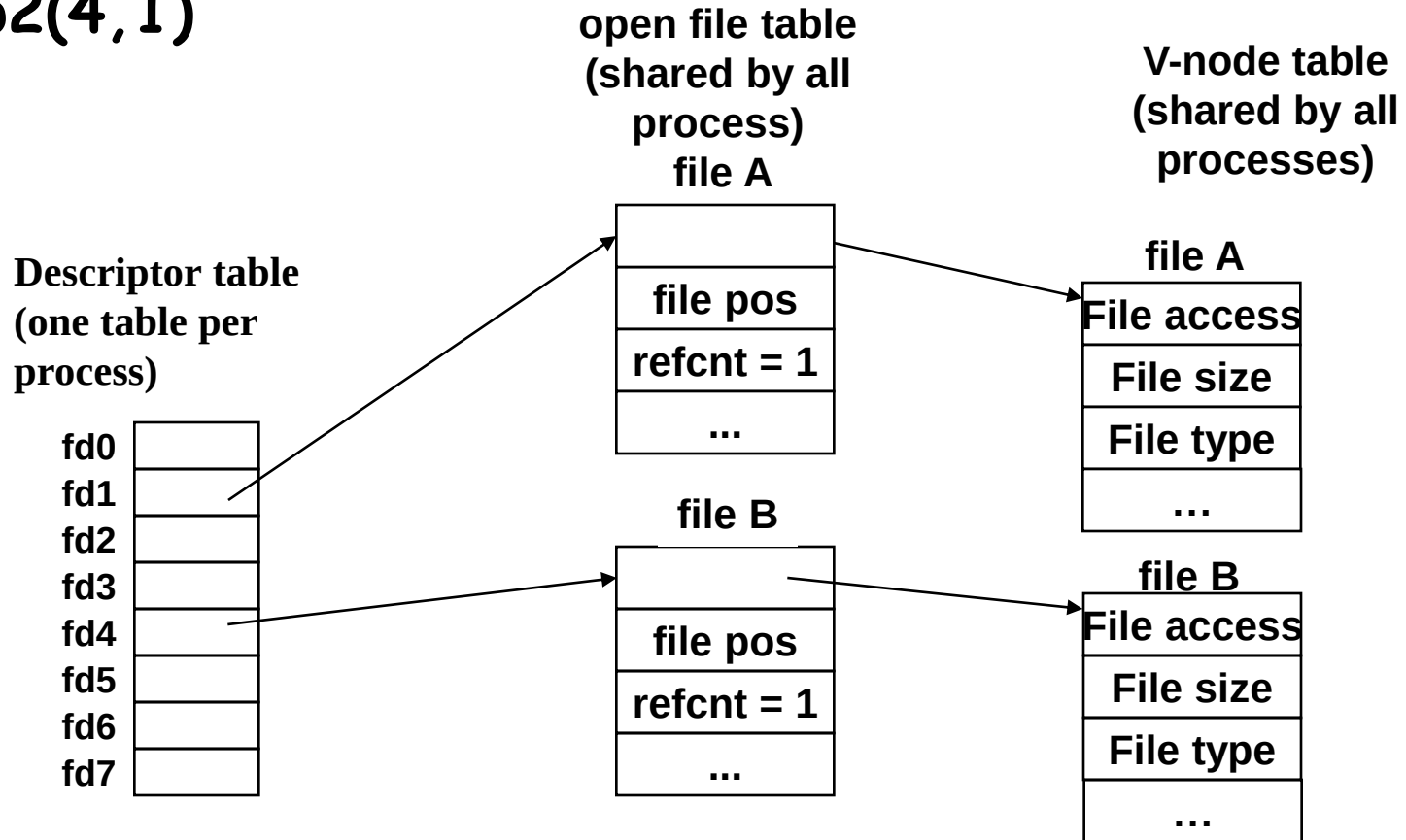
```
int dup2(int oldfd, int newfd);
```

**returns: nonnegative descriptor if OK,
-1 on error**

- `dup2(oldfd, newfd)` **overwrites** the entry in the per-process file table for **newfd** with the entry for `oldfd`.

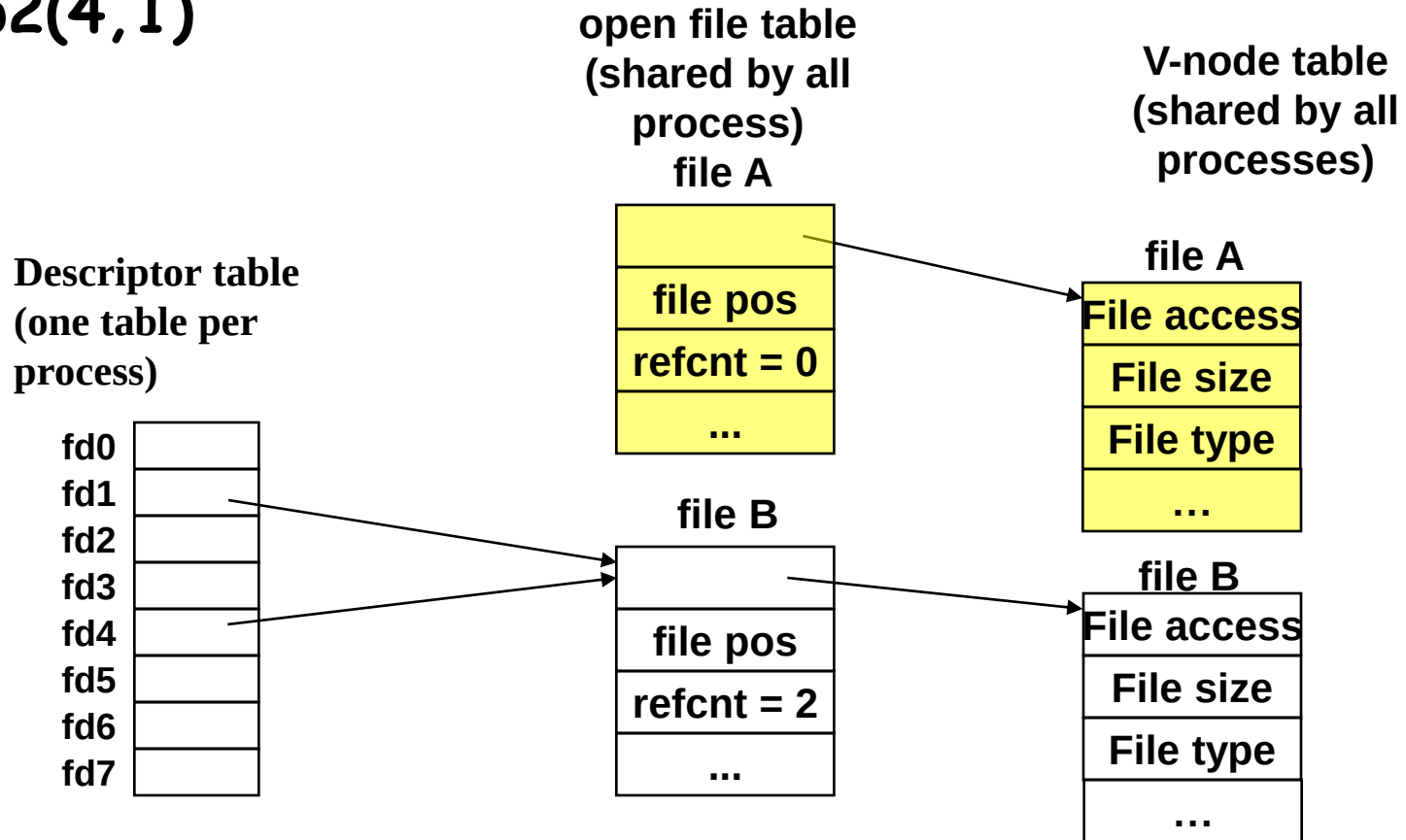
Redirection

`dup2(4,1)`



Redirection

`dup2(4,1)`



Redirection

```
#include "csapp.h"
```

```
int main()
```

```
{
```

```
    int fd1, fd2;
```

```
    char c;
```

```
    fd1 = open("foobar.txt", O_RDONLY, 0) ;
```

```
    fd2 = open("foobar.txt", O_RDONLY, 0) ;
```

```
    read(fd2, &c, 1) ;
```

```
    dup2(fd2, fd1) ;
```

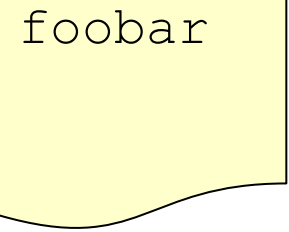
```
    read(fd1, &c, 1) ;
```

```
    printf("c = %c\n", c) ;
```

```
    exit(0)
```

```
}
```

foobar.txt



foobar

屏幕输出什么？